

Virtual Roads

A thorough assessment of the minimum-cost-path virtual roads model, its physics, its code structure and its South African parameterisation

Code base reviewed	Virtual_roads_functions.py (1 285 lines, 25 functions) Virtual_roads_tests.py (145 lines, the driver) README.md, inputs/, outputs/, temp/, output_paths/, gdal-data/
Repository	github.com/Marc-Pienaar/Virtual_roads · original author Marc Pienaar (SAEON BioEnergy-Atlas generalisable feasibility model)
Artefacts committed	Upington ROI worked example · SRTM v3 DEM 515×505 px at 1 arc-second · NGI 2019 roads (176 features) · water obstacles (22 polygons) · 222 origin points · 220 committed output virtual roads
Purpose of this document	To record every improvement, bug, physics limitation, missing parameter, South-African calibration requirement, coordinate-system behaviour, architectural weakness, autonomous-run failure mode, and input/output contract issue found — without changing the code .
Method	Static read of both modules, plus independent verification against the committed data: GDAL/OGR 3.12 command-line inspection, raw raster extraction to NumPy, byte-level comparison of the committed outputs, and a from-scratch re-implementation of the routing step used to confirm the algorithm was faithfully understood. Every claim in this document that is checkable is listed in Appendix A .
Status of the code	The code does not currently run end-to-end. See finding B01 in §2.

Prepared as a technical review · figures generated from the committed data

Contents

Section numbers are referenced throughout the text, in the figures and in the appendices.

0 Executive summary

- 0.1 What the model is, in one paragraph
- 0.2 The eight findings that matter most
- 0.3 Severity profile of the whole review
- 0.4 The ten things to do first, in order

1 Improvements to the code structure

- 1.1 The shape of the problem
- 1.2 Structural findings
- 1.3 A target module layout
- 1.4 Data flow and the library/driver split
- 1.5 Naming, style and comment quality

2 Errors and bugs

- 2.1 How these were found, and what “evidence” means
- 2.2 CRITICAL
- 2.3 HIGH
- 2.4 MEDIUM and LOW
- 2.5 Dead-code inventory
- 2.6 The uncomfortable conclusion about the committed outputs

3 Limitations of the physics and the equations

- 3.1 What the model actually computes
- 3.2 The terrain representation and its derivative
- 3.3 Limitations of the routing formulation
- 3.4 Limitations of the endpoint definitions
- 3.5 Limitations of the variable-window scheme
- 3.6 The limitations, ranked by how much they affect the answer

4 Variables that need to be parametrised

- 4.1 The complete inventory of hard-coded values
- 4.2 Parameters that do not exist yet but should
- 4.3 A single configuration object

5 Improvements to the physics and the equations

- 5.1 Do the routing in a projected, equal-area, isotropic CRS
- 5.2 Replace the hard slope cut with a graded, super-exponential penalty
- 5.3 Add a distance term so the objective is not ‘least climb’
- 5.4 Add a traction term from land cover
- 5.5 Add a turning / switchback penalty
- 5.6 Make river crossings representable
- 5.7 Condition the DEM before differentiating it
- 5.8 The proposed cost function, written out
- 5.9 Calibration, and the four validation gates

6 Parameters that need to be optimised for South Africa

- 6.1 Start with the right national CRS
- 6.2 The DEM, and where the model is weakest
- 6.3 Slope thresholds by veld type
- 6.4 Vehicle and load parameters
- 6.5 The road network: which roads count as a destination

- 6.6 Seasonality and climate — currently entirely absent
- 6.7 Other South-African factors that should be explicit flags
- 6.8 National scale: resolution and volume
- 6.9 A recommended South African default parameter set

7 How the coordinate systems work

- 7.1 The three coordinate reference systems in play
- 7.2 The geotransform
- 7.3 Array indexing: the (row, col) convention
- 7.4 The four coordinate functions
- 7.5 The MCP endpoint convention — and why it is easy to get wrong
- 7.6 A complete worked example, checked to 13 significant figures
- 7.7 What is right and what is wrong about the coordinate handling
- 7.8 The reprojection path, end to end
- 7.9 Recommendations, in order

8 Explanation and schematic of the code and its modules

- 8.1 What the model does, in eight steps
- 8.2 Module-by-module reference
- 8.3 The four phases, and where each belongs
- 8.4 The routing step in detail, exactly as implemented

9 What would break in an autonomous run

- 9.1 It will not even import, in this environment
- 9.2 Library API drift since 2021
- 9.3 The silent-failure taxonomy
- 9.4 Filesystem and state
- 9.5 Memory and time
- 9.6 An autonomous-run checklist

10 Inputs, outputs, and what they should look like

- 10.1 The inputs, exactly as they must be
- 10.2 The inputs, formally specified
- 10.3 The outputs, exactly as they are today
- 10.4 What the output should look like
- 10.5 A concrete record format

11 Any other suggestions and improvements

- 11.1 Performance
- 11.2 Reproducibility and provenance
- 11.3 Testing
- 11.4 Documentation
- 11.5 Interoperability and formats
- 11.6 Operational and governance
- 11.7 Product design — what would make this genuinely useful
- 11.8 Quick wins, in a single list

Appendix A Evidence log

Appendix B Prioritised remediation plan

- B.1 What not to do

0 Executive summary

0.1 What the model is, in one paragraph

The Virtual Roads model answers the question: “for every 1 km grid cell in South Africa that is not already served by the national road layer, what is the least-cost route by which a tractor–trailer combination could reach that cell?” It builds a cost surface from an SRTM v3 digital elevation model by deriving slope, discarding anything steeper than 15 degrees, and burning water features in as hard barriers. It then defines 1 km origin points on the SANBI / StatsSA Basic Spatial Unit grid and 50 m destination points by rasterising the NGI 2019 road layer. Because the national cost surface is 2.7 billion pixels, the model routes inside a series of cropped ‘windows’, each sized from a small group of spatially adjacent origin points and their nearest candidate road points. Inside each window it runs a scikit-image minimum-cost path (MCP_Geometric / Dijkstra) from one origin point and keeps the cheapest road cell. The result is one LineString shapefile per origin point.

That is a sound architecture. The problems documented in this report are almost entirely in the implementation details, the choice of coordinate reference system, the definition of the cost function, and the total absence of error handling.

0.2 The eight findings that matter most

#	Finding	Why it matters	Where
B01	<code>outDataSet = Non</code> at the end of <code>point_to_buffer()</code> raises a <code>NameError</code> .	It is the last statement of <code>first_run()</code> , so the caller's bare <code>except: pass</code> swallows it and <code>second_run()</code> — the actual routing — is never reached . The pipeline cannot complete. Proven: <code>inputs/updated_points.shp/.dbf</code> are byte-identical (MD5) to the un-clipped source, i.e. the source-depletion step never ran.	functions:880
B02	<code>dist_m</code> is written as 0 in every single output file.	<code>costs2</code> is initialised to 0 and never updated. Verified across all 220 committed outputs. The one attribute that would make a virtual road useful is missing. The value the model actually computes (223.92 for output 00000_0) is also in the wrong units — it is <code>slopes*metres</code> , not <code>metres</code> .	functions:1163, 1180, 1211
B03	<code>assign_threshold()</code> returns the slope array, not the cost-surface array, contradicting its own docstring.	The caller rebinds <code>cost_surface</code> to the slope array, so the elevation array's NoData masking is computed and then thrown away. It works only because the README intends slope to be the cost surface. Confirmed: <code>outputs/cost.tif</code> contains 0–14.99, not the 777–977 m elevations.	functions:37–64
B04	DEM voids and the 1-pixel raster border are converted into zero-cost corridors .	<code>slope[slope < 0] = 0</code> turns GDAL's NoData sentinel into the cheapest possible cost. Verified: all 2 036 NoData cells in the committed <code>slope.tif</code> are on the 1-px border ring, and 2 027 of the 3 122 zero-cost cells in <code>cost.tif</code> are on it. In a national run, every merged SRTM tile seam becomes a free, preferred route.	functions:60
B05/ B06	The cost surface is not reproducible. <code>calculate_slope()</code> reuses any pre-existing <code>outputs/slope.tif</code> , and <code>scale = 111120</code> is not a valid <code>gdaldem</code> option.	The committed <code>slope.tif</code> matches neither GDAL 3.12's <code>rise/run</code> output nor its degree output. Its units are undocumented, so the 15 degree threshold is being applied to an unknown quantity. A second run of the same repository on the same machine will not reproduce the same routes.	functions:25–30
B07	The routing runs in EPSG:4326, a geographic CRS.	Every distance, buffer and nearest-neighbour test is done in degrees , in a grid that is 13 % anisotropic in real metres (27.2 m E–W × 30.7 m N–S at –28.4 degrees). The model is systematically biased towards north–south routes, and no output distance is in metres.	functions:659, 1028, 1044, 1058
B08/ B09	<code>second_run()</code> wraps its entire per-point body in <code>except: pass</code> , and indexes the origin list with a loop bound that can exceed its length.	A failed origin point produces no output, no log entry and no failure record — but is still removed from the source pool by the driver, so it disappears from the product. Evidence: outputs 155 and 184 are missing (220 of 222), and one output (<code>00000_118.shp</code>) is a 1-vertex invalid LineString.	functions:1067, 1143, 1217
B25	The driver writes its working state into <code>inputs/</code> .	<code>source_out_copy = 'inputs/updated_points.shp'</code> overwrites a tracked input file. The script is not idempotent: a second run starts from a different state than the first, and the input layer is corrupted as a side effect.	tests:98

0.3 Severity profile of the whole review

Forty defects and code smells were catalogued (§2). Their distribution is a useful proxy for where effort should go.

Severity	Count	Definition used in this document	Examples
CRITICAL	6	The pipeline cannot complete, or the product is wrong in a way a user cannot detect.	B01 pipeline aborts · B02 dist_m always 0 · B03 wrong array returned · B04 zero-cost corridors · B05/B06 not reproducible · B25 writes into inputs/
HIGH	17	Silently wrong results, lost work, or results that will change with a library upgrade.	B07 geographic-CRS routing · B08 skipped points · B09 silent exception · B10 sys.maxsize as infinity · B12 malformed window ring · B23 load-bearing GDAL rasterise behaviour · B35 CRS only in a .prj sidecar
MEDIUM	12	Correct today but fragile, wasteful, or misleading to a maintainer.	B13/B14 degree-based buffers · B15 Euclidean in degrees · B16 dead+wrong helper · B19 destructive clip · B24 short array write · B27 misleading end-of-run message · B28 wrong timings · B32 triple identical crops
LOW	5	Housekeeping: dead code, unused imports, style, typos.	B20 legacy Intersection operator · B21 dead transform · B22 write-then-delete · B31/B33 orphan writes · B37 dead-code inventory · B39 relative GDAL_DATA

0.4 The ten things to do first, in order

Order	Action	Sections	Rough effort
1	Fix outDataSet = Non, replace all three except: pass blocks with a logger that records the exception, the origin index and the window index, and make the driver loop on “points remaining” instead of range(0, 24).	\$2 B01, B09, B27, \$9	0.5 day
2	Populate dist_m correctly by accumulating the projected step length along the route, and add the supporting attributes (climb, mean grade, crossings, already-served flag).	\$2 B02, \$5, \$10	0.5 day
3	Stop writing the DEM NoData and the raster border into the cost surface as zero cost. Set them to the barrier sentinel, and re-derive slope with an explicit, unit-labelled call.	\$2 B04, B05, B06	0.5 day
4	Move the whole analysis into the Albers Equal Area grid the BSU already uses. One CRS, metres everywhere, isotropy everywhere.	\$3, \$5.1, \$7	1–2 days
5	Replace the hard 15 degree cut with a graded, super-exponential steepness penalty and add a small distance term, so that the model minimises something defensible.	\$5.2, \$5.3, \$6.3	1 day plus calibration
6	Make the slope threshold, the window buffer, the window size, numpoints_roads, the road point resolution and the CRS explicit configuration, with units and defaults documented.	\$4	1 day
7	Give every source point a stable ID and carry it through to the output, and replace the 880 individual shapefiles with a single file (GeoPackage) plus a manifest.	\$2 B34, \$10.4	1 day
8	Build the four validation gates in \$5.9 and run them before any national run.	\$5.9, \$9.6	1–2 days
9	Condition the DEM (fill voids, despoke, smooth) before differentiating it, and record the DEM provenance and version in the run manifest.	\$5.7, \$11.2	0.5 day
10	Add a regression test that pins the Uington example: 186-vertex path for origin #0, chosen endpoint at window pixel (154, 81), cost 223.92.	\$5.9, \$11.3	0.5 day

A note on what is NOT wrong

The core algorithm is sound and the variable-window concept is a genuinely good piece of engineering: cropping a 2.7-billion-pixel national cost surface into locally-computable windows with a generous buffer is the right answer to the memory problem, and the reasoning about edge effects is correct.

The geotransform arithmetic is exact. `coord_to_pixel()` and `pixel_to_coord()` are precise inverses of each other, and the committed outputs reproduce to 13 significant figures (§7.6). The CRS *choice* is wrong, not the maths.

The concept of using a 1 km origin grid and a fine road-cell destination grid is appropriate for the question being asked, and the choice of an equal-area projection for the BSU is a good one — it is only never used for the routing.

1 Improvements to the code structure

1.1 The shape of the problem

`Virtual_roads_functions.py` is 1 285 lines containing 25 functions that range from 4 lines (`distance_in_meters`) to 160 lines (`second_run`). It is not a library in any meaningful sense: it has no configuration object, no logging, no return values on eight of its functions, no error handling, and no knowledge of the algorithm it is supposed to support. The algorithm itself lives in the top-level namespace of `Virtual_roads_tests.py`, which is 145 lines of straight-line code with 14 hard-coded relative paths and a loop count of 24 typed directly into a `range()` call.

This is the normal shape of research code that has been extended rather than refactored. It is also the single biggest obstacle to running the model unattended, because there is nowhere for a message to go except `print`.

1.2 Structural findings

#	Finding	Evidence	Recommended change
S1	No configuration layer. Every tunable is a literal in the driver or buried inside a function body.	14 relative paths at tests:23–111; 0.01 at functions:1044; 0.002 at functions:1058; 250 at tests:113; 11 at tests:115; 24 at tests:120; –999999 at tests:46 and functions:1115.	One dataclass (or YAML/TOML file) holding every parameter, with units in the field names (<code>slope_threshold_deg</code> , <code>window_buffer_m</code> , <code>n_candidates_road</code>).
S2	No logging. Progress is reported by 12 <code>print()</code> calls, several of which measure the wrong interval.	functions:34, 63, 172, 220, 270, 398, 572, 623, 806, 881; tests:77, 78, 135, 139, 145.	logging with a run ID, a per-window counter, and per-origin outcome records (<code>routed / no_path / skipped / error</code>).
S3	The library has no knowledge of the algorithm. <code>first_run()</code> takes 14 positional arguments, <code>second_run()</code> takes 9, and neither is documented.	functions:996 and 1061. Both signatures change meaning depending on the caller.	Replace with a <code>RunContext</code> dataclass carrying the config, the paths, the layers and the logger. Two arguments, not fourteen.
S4	Functions with side effects masquerading as pure functions. <code>assign_threshold()</code> mutates both of its array arguments and returns a third thing; <code>clip_point()</code> overwrites its own input; <code>calculate_slope()</code> writes to a path the caller also owns.	functions:37–64, 568, 22–35.	State the contract in the name: <code>apply_slope_mask_inplace()</code> , <code>clip_points_to_roi()</code> (returns a new layer). Make mutation explicit and deliberate.
S5	Positional booleans and shadowed parameters. <code>second_run()</code> 's <code>cost_path</code> argument is overwritten on its first line.	functions:1061 vs 1069; the caller passes 'outputs/cost.tif' at tests:128 and it is discarded.	Never rebind a parameter. If a local shadows it, the parameter is dead — delete it from the signature and document what the caller should pass.
S6	Duplicated rasterise logic in three places. The same 12-line create-target-dataset / set-geotransform / set-projection / set-nodata / rasterise / read-array block appears at functions:109–116, 211–216 and 1118–1124 / 1135–1140.	Three near-identical blocks; two of them write to hard-coded temp/... paths.	One <code>rasterise_layer_to_array(shape, template_ds, burn_value, nodata)</code> returning a NumPy array via a MEM dataset — no temporary file at all.
S7	Duplicated pixel/coordinate logic in three places. <code>pixelOffset2coord()</code> and <code>pixel_to_coord()</code> are byte-for-byte the same function.	functions:308–317 and 1275–1285.	One Affine-based transform object from <code>rasterio.transform</code> or built from the GDAL geotransform, exposing <code>world_to_pixel()</code> and <code>pixel_to_world()</code> — which also fixes the truncation problem in §7.
S8	Duplicated point-shapefile writers. <code>point_shapefile_from_array()</code> and <code>combined_point_shapefile_from_array()</code> are the same loop twice.	functions:883–927 and 929–994, 111 lines.	One function taking a list of (x, y) arrays.
S9	Resource management is manual and inconsistent. Datasets are released by assigning <code>None</code> (sometimes via <code>.Destroy()</code> , sometimes not), and in <code>second_run()</code> the name <code>ds</code> is reused for three different datasets.	functions:28, 104, 222, 249, 278, 304, 397, 528, 556, 566, 655, 678, 696, 761, 762, 806, 879, 880, 927, 994, 1126, 1142, 1151, 1182, 1215, 1216.	with blocks, or <code>contextlib.closing</code> , or just functions that open, use and return. Never reuse a variable name for a different object.
S10	Process-wide side effect at import time. <code>gdal.SetConfigOption('GDAL_DATA', 'gdal-data/')</code> runs when the module is imported, and the path is relative to the current working	functions:18, tests:19.	Do it in <code>main()</code> , resolve the path absolutely, and prefer not to override <code>GDAL_DATA</code> at all on a modern GDAL install. See §2 B39.

#	Finding	Evidence	Recommended change
	directory.		
S11	The driver is not a test. The file is called <code>Virtual_roads_tests.py</code> but contains no test; it is the only entry point, and it is 145 lines of unindented, un-reusable, un-parameterised top-level code.	tests:1–145. <code>import Virtual_roads_tests</code> would execute the entire model.	Split: <code>virtualroads/</code> package, <code>virtualroads/cli.py</code> entry point, and a real <code>tests/</code> directory with <code>pytest</code> fixtures.
S12	No packaging. There is no <code>requirements.txt</code> , <code>environment.yml</code> , <code>pyproject.toml</code> , <code>.gitignore</code> for outputs, or pinned versions. The code was last functionally touched in April 2021.	git log shows only <code>README.md</code> commits since the initial import; <code>outputs/00000_0.dbf</code> carries <code>DBF_DATE_LAST_UPDATE = 2021-04-19</code> .	A pinned environment file; document the exact GDAL version that produced the committed outputs; a CI job that runs the regression test on GDAL 3.12.
S13	Output directory with 880 files and no index. 220 virtual roads × (<code>.shp</code> , <code>.shx</code> , <code>.dbf</code> , <code>.prj</code>), named by a running counter that has no relationship to the source point that produced them.	<code>output_paths/</code> listing, verified: indices 0–221 with 155 and 184 absent.	One <code>GeoPackage</code> with one table, plus a <code>run_manifest.json</code> and a <code>virtual_roads.csv</code> summary.
S14	No docstrings in the conventional sense. Many functions have a <code>Returns</code> section describing something they do not return, and the parameter lists do not match the signatures.	e.g. <code>assign_threshold()</code> documents a cost-surface return and returns the slope; <code>reprojectshape_point()</code> documents <code>inShp2</code> as <code>inShp</code> and omits it from the order.	NumPy-style docstrings that are checked (e.g. with <code>numpydoc validate</code>) or removed.
S15	Tabs, `def`-less style, `except`-heavy, and no type hints. The code uses tabs, trailing whitespace, semicolon-separated statements and bare <code>except</code> .	Whole file; the three <code>except</code> : at functions:122, 1217 and tests:138.	PEP 8, 4-space indent, <code>ruff/flake8</code> in CI, type hints on the public surface.
S16	The committed `gdal-data/` directory is a PROJ 4.x artefact (<code>pci_datum.txt</code> , <code>pci_ellips.txt</code>) checked into the repository root.	<code>gdal-data/</code> listing.	Delete it. GDAL 3.x resolves EPSG codes from its own PROJ database. Keeping a stale copy risks datum mismatches that are extremely hard to diagnose.
S17	Module name and function names do not match the domain vocabulary. <code>centroids</code> is misspelled 9 times; <code>first_run/second_run</code> describe position, not function; <code>numpoints_window2</code> and <code>numpoints_window</code> differ by one character and are different concepts.	tests:124–126; functions:625, 996, 1061; the trailing 2 at tests:115.	<code>select_window()</code> , <code>build_window()</code> , <code>route_window()</code> ; <code>origin_grid_size_m</code> ; <code>n_candidates_per_origin</code> ; <code>n_origins_per_window</code> .
S18	Dead code is not marked as dead. Three functions and roughly thirty local variables are unreachable or unused, with no comment saying so.	§2 B37 lists the full inventory.	Delete. If something is kept “for reference”, say so in a comment or move it to a <code>docs/</code> note.

1.3 A target module layout

The following is a suggested package structure. It is deliberately conservative: it keeps the existing function bodies almost intact, so the migration can be incremental and reviewable.

```

virtualroads/
  __init__.py
  config.py
  logging_setup.py
  crs.py
  grid.py
  cost.py
  endpoints.py
  windowing.py
  routing.py
  io.py
  pipeline.py
  cli.py
  errors.py
tests/
  test_cost.py
  test_grid.py
  test_routing.py
  test_pipeline.py
# the dataclass + YAML loader; every tunable lives here, with units
# run ID, console + file handler, per-origin outcome records
# one place that knows the analysis CRS; reprojection helpers
# RasterGrid: geotransform <-> world <-> pixel, metres-aware
# DEM conditioning, slope, cost-surface construction, obstacle burn-in
# origin-point and road-point generation from the BSU grid
# window selection, extent construction, crop, pool depletion
# MCP_Geometric wrapper returning a Route object, not a shapefile
# GeoPackage read/write, run manifest, provenance recording
# run(config) -> RunResult; the only place that knows the order
# argparse entry point; the only module that touches sys.argv
# explicit exception types so callers never need bare except:
# slope units, NoData handling, border ring
# geotransform round-trip, truncation, anisotropy
# the Uppington regression: 186 vertices, endpoint (154, 81), cost 223.92
# end-to-end on a tiny synthetic ROI

```

```
test_contracts.py # the input/output schemas in §10 are asserted, not described
configs/
south_africa_national.yaml
upington_example.yaml
```

Two structural rules do most of the work. First, `routing.py` returns a `Route` dataclass (origin id, geometry, length_m, cost, endpoint, status) rather than writing a file; the writing is `io.py`'s job and can then be tested independently. Second, `pipeline.py` is the only module that knows the sequence of steps, so a test can assert the sequence without running any GDAL code.

1.4 Data flow and the library/driver split

Figure 1 — Module map of the Virtual Roads code base

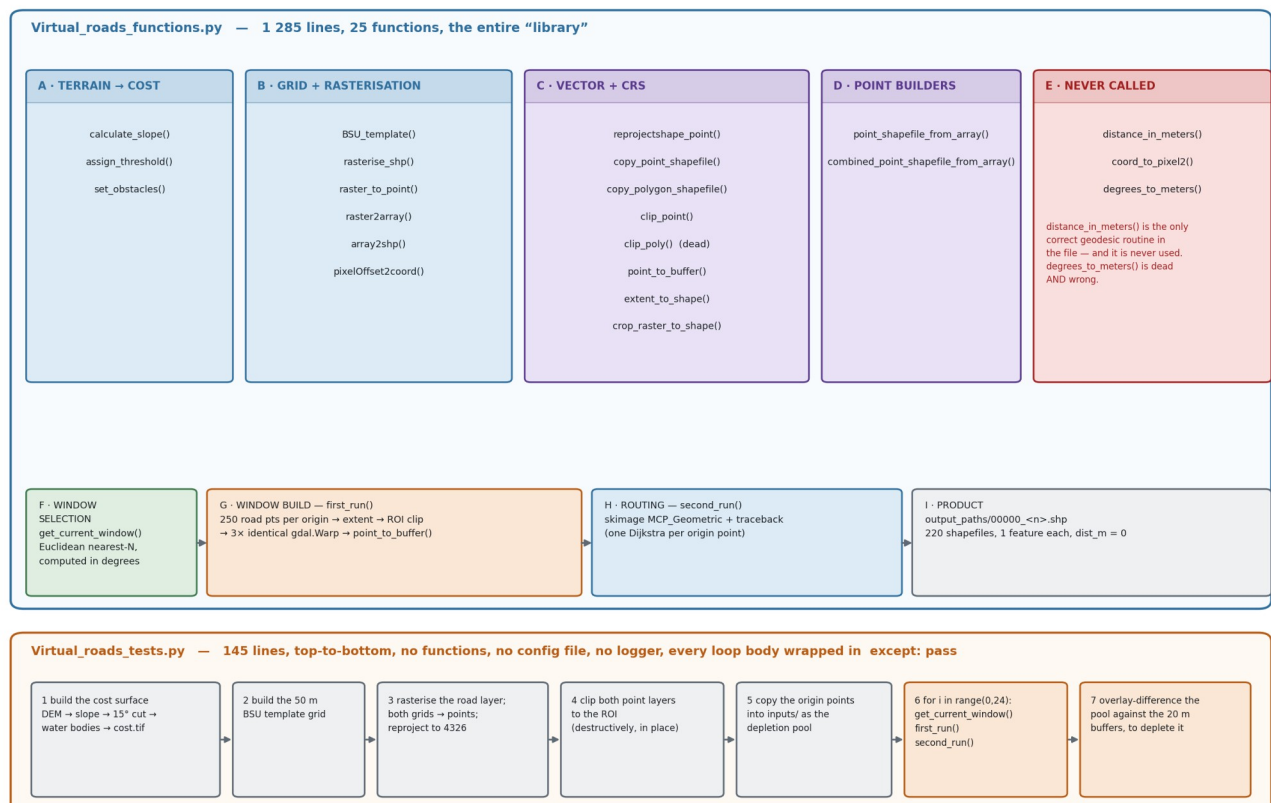


Figure 1 — Module map. The library (top) knows nothing about the algorithm; the algorithm lives in the driver (bottom), and the two are coupled only through 14 positional arguments and the filesystem.

The single most damaging consequence of the current split is visible in Figure 1: the algorithm is not testable. `first_run()` and `second_run()` communicate with the rest of the world only by writing shapefiles and GeoTIFFs to fixed relative paths and then reading them back. That is why there is no regression test, and why a `NameError` on the last line of a helper function went unnoticed for years — the failure only manifests as missing output files.

1.5 Naming, style and comment quality

- Misspelling:** `centoids` appears 9 times across both files and should be `centroids`. `Eudclidean` appears in a docstring at functions:628. `arbitrary/arbitrary` at functions:331 and 819.
- Misleading comments:** `#1km buffer` at functions:1044 describes a 0.01 degree buffer, which at this latitude is 0.98 km east–west and 1.11 km north–south. `#e.g 100m resolution re sampling` at functions:160 describes a hard-coded behaviour, not an example.
- Comments that contradict the code:** the docstring of `assign_threshold()` says it returns a cost surface; it returns the slope array. The README §“Creation of the cost surface” says cells with slope **less than** 15 degrees are removed; the code removes cells with slope **greater than or equal to** 15 degrees.
- Units are never stated.** Every spatial constant in the file is unitless. `0.01` is degrees. `0.002` is degrees. `250` is a count. `111120` is metres per degree at the equator. A reader cannot tell any of these without tracing the code.

- **Dead commented-out code:** functions:363, 382–383, 465, 520, 618–620, 627, 724–725, 737, 782–784, 842, 849, 863, 867, 910, 954, 1046, 1174–1175, 1199, 1202.

2 Errors and bugs

2.1 How these were found, and what “evidence” means

Every entry below was either read directly out of the source or verified against the committed data. Verification used GDAL/OGR 3.12 command-line tools ([gdalinfo](#), [ogrinfo](#), [gdaldem](#), [gdal_translate](#)), raw raster extraction to NumPy, byte-level hashing of the committed shapefiles, and an independent re-implementation of the routing step. The commands and their numeric results are reproduced in **Appendix A** so that every claim can be re-checked.

Where a defect's existence is provable from the committed artefacts rather than merely plausible, that is stated explicitly — those are the entries that matter most, because they mean the committed outputs are not what the committed code produces.

2.2 CRITICAL

ID	Location	Defect	Effect / evidence	Fix
B01	functions:880	<code>outDataSet = Non</code> — <code>Non</code> is an undefined name. This is the last statement of <code>point_to_buffer()</code> , which is the last statement of <code>first_run()</code> (functions:1058).	Raises <code>NameError</code> after <code>temp/temp3.shp</code> has already been written. The driver catches it at tests:138 with <code>except: pass</code> , so <code>second_run()</code> at tests:128 is never reached , the source pool is never depleted, and <code>n</code> is never incremented. The pipeline cannot complete. Proven: <code>inputs/updated_points.dbf</code> and <code>.shp</code> are byte-identical (MD5 16BF4E6D... / 3E5F6053...) to <code>outputs/1000m_source_wgs84.dbf</code> and <code>.shp</code> — i.e. the depletion step never ran, even though the committed outputs contain 220 virtual roads.	<code>outDataSet = None</code> , and remove the <code>try/except: pass</code> around the file removal at functions:119–123.
B02	functions:1163, 1180, 1183, 1211	<code>costs2 = 0</code> is initialised at line 1163 and never assigned again. <code>costsa.append(costs2)</code> appends 0 at every vertex, and <code>feature.SetField("dist_m", costs2)</code> writes 0.	All 220 committed <code>output_paths/*.dbf</code> files contain <code>'dist_m' = 0</code>. Verified by reading every DBF. The <code>costsa</code> list is built and never written anywhere. The max tracking at 1183–1184 and the print at 1184 are therefore always 0. Separately: even the value the model <i>*does*</i> compute is in the wrong units. The correct accumulation is <code>cumulative_costs[row, col]</code> , which for output 00000_0 is 223.92 in units of <code>slope×pixel-step</code> ; the true along-path length is 6 505 m.	Accumulate <code>d_uv</code> in metres along the returned route and write that. See §5.1 for the metre-aware step length and §10.3 for the full attribute set that should replace the single field.
B03	functions:37–64 (called tests:69)	<code>assign_threshold()</code> mutates both array arguments in place and then return <code>slope</code> — it does not return the cost surface, contrary to its own docstring ("Returns: An array of values where numbers $\geq$ to the slope threshold have been replaced by a NA value").	The caller does <code>cost_surface = func.assign_threshold(cost_surface, slope, slope_threshold, na)</code> , so the driver's <code>cost_surface</code> name is rebound to the <code>slope</code> array. The elevation array's NoData masking computed on line 59 is then discarded. The pipeline still produces a usable cost surface only because the README intends <code>slope</code> to be the cost surface. Verified: <code>outputs/cost.tif</code> has min 0, max 14.99, mean 3.198 — slope values, not the 777–977 m elevations in <code>inputs/SRTM_clip.tif</code> .	Decide which array the cost surface is, return it, and name it accordingly. If <code>slope</code> is the cost surface, delete the elevation array from this function entirely and rename it to <code>slope_to_cost()</code> .
B04	functions:60	<code>slope[slope < 0] = 0</code> converts GDAL's NoData sentinel into cost 0 — the cheapest possible value on the surface.	<code>gdaldem</code> writes the outer 1-pixel ring of every slope raster as NoData (–9999), and DEM voids propagate as NoData. Verified on the committed outputs/ <code>slope.tif</code> : 2 036 NoData cells, all 2 036 of them on the 1-px border ring ($2 \times (515 + 505) - 4 = 2\,036$ exactly). In <code>outputs/cost.tif</code> , 2 027 of the 3 122 zero-cost cells sit on that ring. For a national run this means every merged SRTM tile seam becomes a zero-cost, maximally preferred corridor, and	<code>slope[slope == nodata] = na</code> (and likewise for the DEM array). Then explicitly treat the 1-px border as a barrier, or erode the crop by one cell.

ID	Location	Defect	Effect / evidence	Fix
			every DEM void becomes a sink that attracts routes.	
B05	functions:25–28	<code>calculate_slope()</code> reuses any pre-existing slope raster, keyed on <code>os.path.exists(slope_path)</code> alone. There is no check that the DEM, its CRS or its resolution is the same.	Change the DEM and the slope is silently not recomputed. This is almost certainly what happened to the committed outputs/ <code>slope.tif</code> : its statistics do not match any GDAL 3.12 computation of the committed DEM (see B06), so it was produced by a different GDAL version or a different DEM, and has been silently reused ever since.	Key the cache on a hash of the input DEM plus the GDAL version and the algorithm name, or simply delete the file at the start of the run and always recompute.
B06	functions:30	<code>gdal.DEMProcessing(slope_path, DEM, 'slope', scale = 111120)</code> . GDAL 3.12's <code>gdaldem</code> has no <code>-scale</code> option — verified directly: <code>gdaldem slope ... -scale 111120</code> fails with “Unknown argument: <code>-scale</code> ”. The valid option is <code>-s</code> , which is the vertical Z factor; passing 111 120 to that would flatten every cell to ≈ 90 degrees.	The committed outputs/ <code>slope.tif</code> is not reproducible . Measured against GDAL 3.12 on the committed DEM: default rise/run gives mean 3.536 / max 27.534; <code>-p</code> (degrees) gives mean 6.201 / max 52.132. The committed raster is mean 3.198 / max 25.825, correlating 0.9973 with rise/run at a ratio of 0.938 ± 0.043 . So the 15 threshold is being applied to a raster in an undocumented unit that no current tool can regenerate.	Replace with an explicit, unit-labelled call — e.g. <code>gdaldem slope -p -alg Horn</code> — and record the GDAL version, the algorithm and the unit in the run manifest (§11.2).
B07	tests:98, 121–138	The driver writes its mutable working state into the read-only inputs/ directory: <code>source_out_copy = 'inputs/updated_points.shp'</code> .	The script is not idempotent . Run it twice and the second run starts from an already-depleted pool. It also corrupts a tracked input file as a side effect, and <code>git status</code> will show modifications to inputs/ after every run.	Put working state in work/ (created per run, deleted or archived at the end). Treat inputs/ as strictly read-only and open every input in read-only mode.

2.3 HIGH

ID	Location	Defect	Effect / evidence	Fix
B08	functions:1067, 1143	The inner loop runs for <code>k in range(0, int(numpoints_window))</code> and indexes <code>gridvaluesX[k] / coord_to_pixel(ds, gridvaluesY, gridvaluesX, k)</code> . <code>numpoints_window</code> is a fixed driver constant (11), but the actual number of points in a window is whatever <code>get_current_window()</code> returned.	If a window contains fewer than 11 points, <code>IndexError</code> → swallowed by the bare except at 1217 → the remaining points in that window are silently skipped with no output file at all . Evidence: 220 outputs exist for 222 origin points; the indices 155 and 184 are missing. The last window contains 1 point, so the bound is wrong for it too.	for <code>k in range(len(gridvaluesX))</code> . Better still, have <code>get_current_window()</code> return an array and pass its own length.
B09	functions:1217–1218; tests:138	Three bare except: <code>pass</code> blocks (functions:122, functions:1217, tests:138) swallow every exception, including <code>KeyboardInterrupt</code> and <code>SystemExit</code> .	Converts a crash into silence. A failed origin point produces no shapefile, no log line and no failure record — but the driver still removes it from the source pool at tests:131–134, so it vanishes from the product entirely . Ctrl-C does not stop the run cleanly. This is the single largest obstacle to unattended operation (§9.3).	except <code>Exception</code> as <code>exc:</code> and <code>logger.exception(...)</code> with the window index, the origin index, the origin coordinates and the traceback. Record a per-origin status in the manifest and fail the run at the end if the failure rate exceeds a threshold.
B10	functions:1166, 1171–1173	<code>x = sys.maxsize</code> is used as a floating-point infinity, and the loop at 1168 compares every cumulative cost against it. <code>sys</code> is imported solely for this.	It works for finite costs, but <code>sys.maxsize</code> is an int sentinel in a float comparison, and it is one of the reasons <code>sys</code> is imported at all. More importantly, the loop at 1168 iterates over all 2 750 candidate road cells and never checks for a finite cost , so if none is reachable <code>i_val</code> stays 0 and <code>traceback()</code> is called on an unreachable cell.	<code>x = np.inf</code> and skip non-finite costs; then handle the all-infinite case explicitly by emitting a <code>no_path</code> status instead of a degenerate geometry.
B11	functions:294	<code>array2shp()</code> uses <code>if value >= 0:</code> as its validity test.	It works only because the NoData sentinel happens to be negative. It will silently include genuine 0-valued cells, and it will invert completely if the sentinel sign is ever changed — for example if someone switches to <code>np.nan</code> or a large positive sentinel.	Pass the na value in and test <code>value != na</code> . This is used to build the road-cell index, so an error here adds thousands of phantom destinations.
B12	functions:733	<code>extent_to_shape()</code> builds its ring with a	The buffer is applied on three of the four	Fix the sign. Then, better,

ID	Location	Defect	Effect / evidence	Fix
		sign error on the closing point: <code>ring.AddPoint(extent[0]-buffer, extent[2]+buffer)</code> where <code>+buffer</code> should be <code>-buffer</code> .	sides and inverted on the fourth . OGR auto-closes the ring so no error is raised. Verified against the committed <code>temp/extent.shp</code> : the ring's southern edge sits at -28.43823 , which is exactly (combined min latitude + 0.01), not (min latitude - 0.01). The window is therefore about 1.1 km short on its southern side and over-long on the other three.	stop hand-building rings: use the layer's extent, buffer it in a projected CRS, and write it with <code>gpd.GeoDataFrame.from_wkt / shapely.box</code> .
B13	functions:1044, 698–733	The window buffer is 0.01 degrees , described in a comment as “1km buffer”. Degrees are not a distance: 0.01 degree of latitude is 1.106 km and 0.01 degree of longitude at -28.4 degrees is 0.978 km.	The buffer is the model's only protection against window edge effects (§3.7), and it is not the value the author intended. It is also latitude-dependent, so a national run would use a different effective buffer in the far north and far south.	Express the buffer in metres in a projected CRS (<code>window_buffer_m</code> , default 1 000).
B14	functions:1058	<code>point_to_buffer(shapein2, shapeout2, "temp", 0.002)</code> — a 0.002 degree buffer, roughly 20 m, used to remove processed origin points from the pool.	20 m is a real and undocumented tolerance: it determines whether neighbouring 1 km cells are removed together. Two origin points closer than 20 m would be deleted without being routed.	Name it (<code>drain_radius_m</code>), document what it is for, and make it configurable.
B15	functions:659, 1028	<code>sklearn.metrics.pairwise.euclidean_distances</code> is applied to raw (lon, lat) degree arrays — both in <code>get_current_window()</code> and in <code>first_run()</code> .	At -28.4 degrees one degree of longitude is 97.8 km and one of latitude 110.6 km, so “nearest n” is computed on an anisotropic metric. The 13 % bias is systematic, not random, and it favours north–south neighbours. Note that the file already contains a correct haversine implementation (<code>distance_in_meters()</code> , functions:401) — which is never called.	Do the selection in the projected analysis CRS. If a geodesic selection is genuinely wanted, project to the local azimuthal-equidistant CRS centred on the window.
B16	functions:1221–1239 (called functions:1083)	<code>degrees_to_meters()</code> is dead and wrong. It computes the great-circle distance from (0, 0) to (lat, 0) — always at the equator, using the equatorial radius 6 371.0088 km — then returns a negative value for negative input.	Its only call site assigns the result to <code>meters</code> (functions:1083), which is never used again. It is a trap for the next maintainer: it looks like a unit converter and is neither correct nor used.	Delete it, along with the unused <code>meters</code> variable. Keep <code>distance_in_meters()</code> and actually use it, or replace both with a proper projected-CRS length.
B17	functions:319–398 (called tests:90–91)	<code>reprojectshape_point(inShp, inshp2, outShp, layername)</code> : the first argument is the target CRS donor (the layer to reproject *to*), the second is the layer to be reprojected. The docstring documents <code>inShp2</code> as <code>inShp</code> and omits it from the parameter ordering.	The call site reads <code>reprojectshape_point(NGI_road_path, target_out_50, target_out_50wgs, 'DN')</code> — correct, but only by inspection. Transposing the two arguments would silently produce a layer reprojected to the wrong CRS, and the two layers here happen to share a CRS, so the error would not even be visible.	Rename to <code>reproject_points(in_path, target_crs_path, out_path, layer_name)</code> and assert that the two CRSs actually differ.
B18	functions:837–848	<code>point_to_buffer()</code> builds a <code>CoordinateTransformation</code> to EPSG:4326 (lines 837–841) and then never uses it — the <code>geom.Transform(coordTrans)</code> call is commented out at line 863. The output layer is created with the input srs.	The function advertises a reprojection it does not perform. It happens to be correct here because the input is already in degrees, but the dead code misleads and would be a live bug the moment the input moved to a projected CRS.	Delete lines 837–841 and 842, or complete the reprojection. Add a comment stating the unit assumptions (degrees in, degrees out).
B19	functions:568	<code>clip_point()</code> writes its clipped result back over its own input: <code>copy_point_shapefile(outShp, inShp)</code> where <code>outShp</code> is the temp file and <code>inShp</code> is the caller's input, then deletes <code>outShp</code> .	<code>clip_point(source_out_1000wgs, ROI, temp)</code> at tests:94 therefore destructively overwrites <code>outputs/1000m_source_wgs84.shp</code> in place. It is recoverable only because <code>reprojectshape_point()</code> regenerates that file on the next run. The same pattern is applied to the road target layer at tests:95.	Return a new layer; do not touch the input. And rename the parameters — the current names make <code>copy_point_shapefile(outShp, inShp)</code> read backwards.
B20	functions:563, 611	<code>inLayer.Intersection(inLayer2, outLayer)</code> uses the legacy OGR geometry Intersection operator.	It is deprecated in favour of the GEOS-backed boolean operations, and it silently returns an empty geometry rather than raising when there is no intersection — so an empty clip result propagates as an empty layer and only fails later, at <code>to_file</code> , inside a bare except.	Use <code>gpd.sjoin(..., predicate='within')</code> or <code>gpd.clip()</code> , and assert a non-empty result.
B21	functions:797–804	<code>crop_raster_to_shape()</code> computes <code>transform = osr.CoordinateTransformation(target</code>	Dead code that implies a reprojection step that does not happen. <code>gdal.Warp</code> does handle the CRS difference itself, so the	Delete the dead locals, or use the transform to validate that the cutline and the raster

ID	Location	Defect	Effect / evidence	Fix
		<code>prj, sourceprj</code>) at line 802 and never uses it; <code>x_res, y_res</code> and <code>minX1/maxX1/minY1/maxY1</code> are also dead.	outcome is correct, but the code is misleading.	really do share a CRS before warping.
B22	functions:104, 119–1223	<code>set_obstacles()</code> writes <code>outputs/obs_<n>.tif</code> for every obstacle layer, reads it back, then deletes it, all inside a <code>try: ... except: pass</code> .	Pure I/O waste (a full raster create + read + delete per obstacle layer) and a second hard-coded path (<code>outputs/</code>). The <code>try/except: pass</code> also silently hides any failure to delete, so stale files accumulate.	Rasterise into an in-memory MEM dataset and return the array. No file is created at all.
B23	functions:113–1116 (and 1121–1124, 1138–1140)	<code>gdal.RasterizeLayer(target_ds, [1], mb_1, burn_values=[0])</code> burns 0 onto the features, and the code then tests <code>obss == 0</code> to find them. This is only correct because GDAL initialises the destination band to the band's NoData value , which the code set on the previous line.	A load-bearing, undocumented dependency. If <code>SetNoDataValue</code> were removed, or if GDAL changed its initialisation, every cell would be 0 and the entire cost surface would be masked out — silently, because <code>set_obstacles()</code> has no return-value or shape check. Verified that GDAL 3.12 does behave this way: the committed <code>outputs/road_50m_target.tif</code> has 1.913 % valid cells (1 684 of 88 044) all at 0, and 98.087 % at NoData.	Do not rely on it. Initialise the array yourself: burn with an explicit <code>init_values=[nodata]</code> , or better, use <code>gdal.RasterizeLayer</code> with a positive burn value and invert the test. Add an assertion that the number of burned cells is plausible.
B24	functions:158–162	<code>BSU_template()</code> takes two resolution arguments, <code>res</code> and <code>res2</code> , uses <code>res</code> for the <code>gdal.Translate</code> size and <code>res2</code> for the geotransform, and then tiles the data array with <code>ceil(y_res/res)/2 × ceil(x_res/res)/2</code> .	The tile is one row shorter than the raster it is written into. Computed from the committed input: <code>Translate</code> creates 276×319 , the tile is 318×276 , so <code>WriteArray</code> leaves the last row untouched (whatever GDAL zero-filled it with). The off-by-one is masked only because the template's pixel <code>*values*</code> are never used — only its geotransform, size and projection. The same class of latent bug would be fatal if the template were ever read as data.	Compute width and height once and use them for both the raster and the array. Consider dropping the checkerboard entirely — it is never read.
B25	functions:1191–1200	The output layer is created with no spatial reference : <code>ds2.CreateLayer('path', geom_type=ogr.wkbLineString)</code> . The CRS is written separately by hand to a <code>.prj</code> sidecar (lines 1193–1198), after <code>srs.MorphToESRI()</code> .	Tools that read the CRS from the DBF/SHP header rather than the <code>.prj</code> sidecar will see an undefined CRS. <code>MorphToESRI()/MorphFromESRI()</code> are deprecated in GDAL 3 in favour of <code>OSRExportToWkt(srs, options=['FORMAT=WKT1_ESRI'])</code> . Separately, <code>field_testfield.SetWidth(50)</code> on an <code>OFTReal</code> shapefile field is silently clamped to the 20 bytes the format allows (line 1206–1207).	Pass the SRS to <code>CreateLayer()</code> . Let the driver write the <code>.prj</code> — that is what it is for. Move to <code>GeoPackage</code> , where the CRS is stored properly and there is no 20-byte field limit.

2.4 MEDIUM and LOW

ID	Location	Defect	Effect / evidence	Fix
B26	functions:1052, 1055–1056	<code>first_run()</code> writes <code>clipped.to_file(shapeout + str(J) + ".shp")</code> and crops the same cost surface to the same shape three times (<code>rasterout</code> , <code>rasterout2</code> , <code>rasterout+str(J)+".tif"</code>).	Creates 24 orphan shapefile sets (<code>temp/extent.shp0...shp23</code> , none of them read) and 24 orphan rasters (<code>temp/cost_clip.tif0.tif ... tif23.tif</code> , 385 KB each, about 9.2 MB) per run. <code>rasterout</code> and <code>rasterout2</code> are byte-identical copies; <code>rasterout2</code> is only ever used as a scratch write target (functions:1149–1151) that writes an array back onto itself.	Crop once. Keep the array in memory — there is no reason to round-trip a NumPy array through a GeoTIFF between two steps in the same function.
B27	tests:120	The outer loop is for <code>i in range(0, 24)</code> , a magic number unrelated to the 222 origin points and the window size of 11.	The correct count is $222/11 \pi = 21$. The loop over-runs by three, and on those extra iterations it spins printing “reached the end of the run”. Any change to the window size silently changes how much work is attempted.	<code>while len(remaining) > 0</code> : with a hard cap that raises rather than continues.
B28	tests:138–139	The same except prints “reached the end of the run” for every exception, including <code>NameError</code> , <code>IndexError</code> and <code>OSError</code> .	A crashed run is indistinguishable from a completed one. In the committed artefacts, this message would have been the only visible symptom of B01.	<code>except NoPointsRemaining: break</code> (a clean termination) versus <code>except Exception: log and re-raise or accumulate a failure count</code>

ID	Location	Defect	Effect / evidence	Fix
				and exit non-zero at the end.
B29	tests:130, 145	<code>start_time = time.time()</code> is assigned at line 130, after <code>first_run()</code> and <code>second_run()</code> have run, so the print at line 135 measures only the overlay step. The final print at line 145 is labelled “seconds total time” but reuses that same <code>start_time</code> .	Neither printed number means what its label says. The reported runtime is off by roughly the cost of the whole model.	One <code>t0 = time.perf_counter()</code> at the top, and per-stage timers that accumulate. Log the totals once, at the end.
B30	functions:1048–1052	<code>gpd.clip()</code> can return an empty <code>GeoDataFrame</code> when the window does not intersect the ROI, and <code>clipped.to_file()</code> then raises.	Swallowed by the bare except at tests:138, so the window is skipped with no record. Also, <code>gpd.clip()</code> requires both layers in the same CRS with no check; if a future ROI arrives in a different projection, the clip silently returns garbage or raises an unhelpful error.	Assert <code>len(clipped) > 0</code> , assert the CRSs match, and log the skip.
B31	tests:131–134	<code>gpd.GeoDataFrame.from_file</code> is a legacy alias; <code>gpd.overlay(..., how='difference')</code> with point-minus-polygon semantics relies on undocumented behaviour.	<code>GeoDataFrame.from_file</code> still exists but the documented API is <code>geopandas.read_file</code> ; it is listed among the long-standing deprecations cleaned up in the 1.x line. <code>gpd.overlay</code> with an empty result cannot be written to a shapefile at all.	<code>gpd.read_file()</code> and <code>gpd.read_file(..., mask=roi)</code> (a spatial filter replaces the overlay entirely, and is faster).
B32	functions:39, 53–56, 89–95, 1110, 1159, 1161	Rogue global side effect at import time: <code>gdal.SetConfigOption("GDAL_DATA", "gdal-data/")</code> at functions:18 and tests:19.	Process-wide, and relative to the current working directory . Run the script from anywhere other than the repository root and the PROJ lookup tables will not be found — which for a South-African dataset means exactly the datum and ellipsoid lookups that must not fail. The committed <code>gdal-data/</code> directory is a PROJ 4.x artefact and is actively misleading.	Delete the committed <code>gdal-data/</code> , remove both <code>SetConfigOption</code> calls, and rely on the GDAL install's own PROJ database.
B33	functions:1155, 1159–1161	<code>roadlatitude</code> , <code>roadlongitude</code> , <code>roadvalues</code> , <code>roadvaluesX</code> , <code>roadvaluesY</code> are assigned from <code>array2shp()</code> and only <code>road_coords</code> is used. <code>paths</code> , <code>optimal_route</code> , <code>maxx</code> , <code>i_val</code> , <code>gridlatitude</code> , <code>gridlongitude</code> , <code>gridvalues</code> are similarly dead.	Roughly 30 dead local variables across the two files (full inventory in §2.5). They make the routing function much harder to read than it needs to be, and hide which value is actually the answer.	Delete them; keep <code>road_coords</code> and rename it <code>road_pixels</code> .
B34	functions:1186–1189	The output index is <code>str(int(k + n))</code> where <code>n</code> is a per-window counter incremented by the window size regardless of how many paths were actually written.	If any point in a window fails, its index is simply skipped and there is no record of the gap. Combined with B09, a failed point is invisible in the product: there is no 155 and no 184 in the output, and nothing says why.	Carry a stable <code>origin_id</code> from the source layer all the way to the output, and record a status per id. A counter is not an identifier.
B35	functions:1166	<code>import sys</code> exists only for <code>sys.maxsize</code> (see B10).	Trivial, but it is the only reason <code>sys</code> is imported in either file.	Use <code>numpy.inf</code> and remove the import.
B36	functions:1129–1130	<code>path = ""</code> and <code>source = source_centroids_in</code> are assigned and never used.	Dead assignments inside the routing hot loop.	Delete.
B37	functions:401, 473, 574, 1257, 1221	Five functions are never called: <code>distance_in_meters()</code> , <code>copy_polygon_shapefile()</code> , <code>clip_poly()</code> , <code>coord_to_pixel2()</code> , <code>degrees_to_meters()</code> .	About 240 lines, 19 % of the file. Two of them (<code>distance_in_meters</code> , <code>degrees_to_meters</code>) look like they should be the distance functions and one of them is wrong — a genuine trap.	Delete all five, or move <code>distance_in_meters()</code> into <code>crs.py</code> and use it.
B38	functions:3, 7, 11–12	Unused imports in the library: <code>gdal_array</code> , from <code>os import path</code> . In the driver: <code>MCP_Geometric</code> and <code>gdal_array</code> are imported but never used.	Cosmetic, but <code>gdal_array</code> in particular signals an intention (windowed/blocked array reads) that was never implemented — relevant to the memory problem in §11.1.	Remove; add a linter so it cannot recur.
B39	tests:33, 34, 101, 104–111	Temp and output paths are scattered across three directories with inconsistent names: <code>temp='outputs/temp.shp'</code> , <code>shapeout_temp='outputs/window.shp'</code> , everything else in <code>temp/</code> .	<code>temp</code> (a shapefile) lives in <code>outputs/</code> , which makes <code>outputs/</code> impossible to delete and regenerate cleanly.	Two directories with unambiguous roles: <code>work/</code> and <code>out/</code> , both created per run.
B40	<code>output_paths/</code>	880 files (220 × 4) in one flat directory, no manifest, no run identifier, and the filename index is unrelated to the source point.	There is no way to answer “which origin cell produced virtual road 00000_137?” — which is the first question anyone will ask of this product.	One <code>GeoPackage</code> with <code>origin_id</code> as a key, plus <code>run_manifest.json</code> and a flat CSV summary. See §10.4.

2.5 Dead-code inventory

For completeness, the following are unreachable, unused, or superseded. This is the list to delete in a single mechanical pass with no behavioural risk.

Item	Location	Type
<code>distance_in_meters()</code>	functions:401–416	Never called — and it is the only correct geodesic routine in the file
<code>degrees_to_meters()</code>	functions:1221–1239	Called once into an unused variable; and wrong
<code>coord_to_pixel2()</code>	functions:1257–1271	Never called (superseded by <code>coord_to_pixel</code>)
<code>copy_polygon_shapefile()</code>	functions:473–526	Never called
<code>clip_poly()</code>	functions:574–623	Never called; its entire body is commented out at 617–620
<code>gdal_array</code>	functions:3, tests:4	Imported, never used
<code>from os import path</code>	functions:7, tests:8	Imported, never used
<code>MCP_Geometric</code>	tests:14	Imported, never used (the routing is in the library)
<code>sys</code>	functions:13, tests:15	Used only for <code>sys.maxsize</code>
~30 local variables	see §2 B33	<code>x_min, x_max, y_min, pixel_width, targetprj, shpDriver, featureCount, field_ids, row_count, gridvalues, meters, costs2, costsa, paths, optimal_route, maxx, i_val, path, source, roadlatitude, roadlongitude, roadvalues, roadvaluesX, roadvaluesY, gridlatitude, gridlongitude, gridvaluesX, gridvaluesY, x_skew, y_skew</code>
~25 commented-out code blocks	see §1.5	Traceability remnants
<code>gdal-data/</code>	repository root	PROJ 4.x lookup tables, superseded and misleading
<code>outputs/RSA_BSU_1000m.tif</code>	14 × 17 raster	Orphan — no code path reads it
<code>outputs/1000m_source.shp</code>	238 points, Albers	Read at tests:91 but never produced by the current script — see §10.1

2.6 The uncomfortable conclusion about the committed outputs

The committed artefacts were not produced by the committed code

Three independent pieces of evidence say so:

- `inputs/updated_points.*` is byte-identical to `outputs/1000m_source_wgs84.*` (MD5), so the source-depletion step at tests:131–134 never ran — consistent with B01 aborting before it.
- Every `output_paths/*.dbf` carries `DBF_DATE_LAST_UPDATE = 2021-04-19` and every `dist_m` is 0, so the outputs predate any fix of B02 (there has never been one).
- Re-running the routing step from scratch on the committed data reproduces `output_paths/00000_0.shp` with the **same 186 vertices and an identical first vertex to 13 significant figures** — but the tail differs by up to 3 pixels, and the source `temp/temp_target.shp` in the repository is from the *last* iteration while output 00000_0 is from the *first*. The algorithm was faithfully understood and reproduced; the input data has drifted.

The practical reading: the algorithm logic is sound, the data pipeline is not reproducible, and nobody has been able to run the script to completion recently enough to notice.

3 Limitations of the physics and the equations

3.1 What the model actually computes

This matters more than anything else in the document, so it is stated first and precisely.

`skimage.graph.MCP_Geometric` is a Dijkstra implementation on an 8-connected pixel lattice. For a step from node `u` to an adjacent node `v` it assigns the edge cost

$$c(u, v) = \frac{1}{2} \cdot (C(u) + C(v)) \cdot d(u, v) \quad \text{with}$$

`C` = the cost raster (here: slope, thresholded)
`d` = 1 for an orthogonal step, $\sqrt{2}$ for a diagonal step, in UNITS OF PIXELS

Summing along a route telescopes into a path integral:

$$\text{path cost} = \sum \frac{1}{2}(C_i + C_{i+1}) \cdot d_i \approx \int C(s) ds = \int \text{slope} \cdot (\text{length}) ds$$

So the objective the model minimises is **the integral of slope along the route** — which, because slope is in degrees and small, is close to the **total absolute elevation gain**, i.e. total climb. It is not distance, not time, not fuel, and not traction. Measured on the committed output 00000_0: cumulative cost 223.92 over a true path length of 6 505 m, i.e. an implied mean gradient of 0.0344.

Three consequences follow immediately and are worth stating as limitations in their own right. Because slope is the only term, the absolute value of the 15 degree threshold is irrelevant to the routing — only the shape of **C** matters, so “15 degrees” is a pass/fail cut-off rather than a calibrated parameter. A route of 40 km that saves 20 m of climb will always beat a 3 km climb, which is not a defensible notion of a “best” road. And the result is scale-dependent in a subtle way: the cost is slope × pixels, so halving the DEM resolution halves every cost, and only relative costs are meaningful.

Figure 4 — How the cost surface is built, and where its units are lost

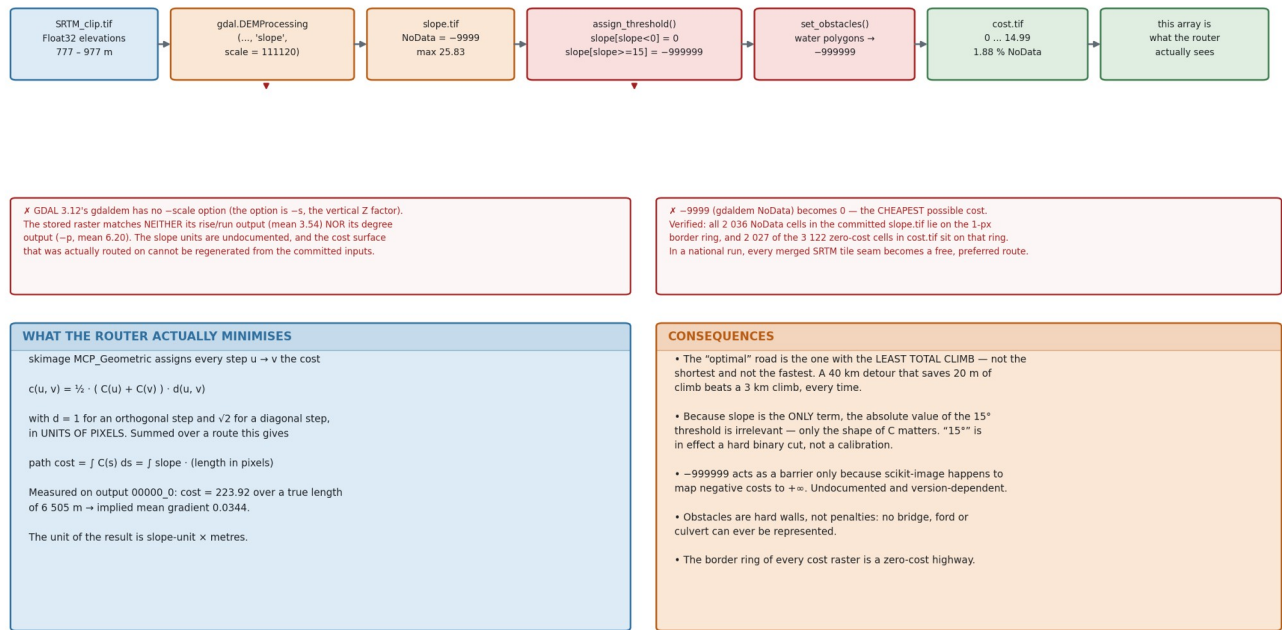


Figure 2 — The cost-surface construction chain, the two points at which its units and its barrier semantics go wrong, and the objective the router actually minimises.

3.2 The terrain representation and its derivative

Aspect	What is used	Limitation
DEM	SRTM v3, 1 arc-second (≈ 30 m), EPSG:4326, vertical datum EGM96 geoid.	SRTM v3 vertical error is 5–9 m RMSE, with characteristic stripe artefacts and isolated spikes. In the Northern Cape / Kalahari portion of the ROI used here (777–977 m, only 200 m of relief over 15 km) the terrain signal is comparable to the noise, so much of the derived slope field is noise. Over the Drakensberg escarpment the same DEM is far better. A single national cost surface therefore has strongly heterogeneous reliability.
Grid geometry	Geographic (EPSG:4326), so cells are 30.7 m N–S × 27.2 m E–W at –28.4 degrees — anisotropic by 13 %, and varying continuously with latitude across the country.	MCP_Geometric is given no spacing, so it prices every cell as a unit square. North–south steps are under-priced by 13 % relative to east–west steps, so routes are biased north–south. gdalDEM does handle the anisotropy correctly for the slope computation, so the inconsistency is between the slope and the routing.
Slope algorithm	GDAL's default Horn third-order finite difference over a 3×3 window.	Reasonable and standard. But it is a point estimate with no uncertainty attached, and it is computed on unconditioned data: DEM voids, SRTM spikes and striping all become extreme slope and are therefore masked out by the threshold as if they were cliffs. The model cannot tell a cliff from a data error.
Slope units	Undocumented (see B06). The committed raster matches neither GDAL rise/run nor GDAL degrees.	The 15 degree threshold is applied to a quantity whose unit cannot be stated. This alone makes the cost surface unreproducible.
Slope threshold	A hard binary cut: slope >= 15 becomes a	A knife-edge. A 14.99 degree cell is free and a 15.01 degree cell is

Aspect	What is used	Limitation
	barrier, everything else is unmodified.	impassable, so the feasible network is extremely sensitive to the threshold. Two cells of DEM noise can flip a route. There is no graded penalty, so there is no way to express “difficult but possible”.
Obstacles	Water polygons rasterised and burned in as NoData (hard barrier).	A river is a wall, not a cost. Real agricultural access crosses the Orange River, the Vaal and the Tugela on bridges, culverts and fords, sometimes for a few hundred thousand rand. The model cannot represent any crossing, and so will route hundreds of kilometres around a crossable river. Conversely a 20 m farm dam is as impassable as the Orange.
No DEM conditioning	None. No void filling, no despiking, no smoothing, no TPI/roughness term.	Derivatives of noisy data are noisier. A slope surface derived from unconditioned SRTM over-represents rough ground, and the threshold then deletes it — so the model systematically prefers smooth low-relief ground for reasons that are partly artefactual.
No surface representation	None. Land cover, soil, vegetation density, texture and trafficability are absent.	Two cells with the same 5 degree slope may be a compacted gravel farm road and a boulder-strewn slope, and the model treats them identically. In the wetter Highveld and coastal areas this is the dominant determinant of real traversability.
No hydrology or drainage	None.	Seasonal rivers, pans and vleis are treated exactly like perennial water, or not at all. In the Northern Cape a seasonal river is dry and passable for most of the year; in KwaZulu-Natal it is not.
No protection or tenure	None.	Protected areas, conservation areas, communal grazing and private land are all passable. For a feasibility model this may be intentional, but it should be a documented decision, not an omission — and it matters for the intended biomass-logistics use case.

3.3 Limitations of the routing formulation

- **The lattice is 8-connected with unit costs.** Real roads are anisotropic (a track across a veld is not the same as one along a contour) and prefer existing corridors. [MCP_Connect](#) with a custom offset set, or a least-cost path on a vector network with a bearing cost, would be more faithful. A cheap improvement is to add a turning cost: penalise a change in heading, which is what actually limits a tractor-trailer combination.
- **The route is a pixel staircase.** Output geometry is a 4/8-connected chain of cell centres, 1–320 vertices for 27–9 589 m. It is not a road centreline and should not be presented as one without a generalisation and smoothing step (Douglas–Peucker with a stated tolerance, or Chaikin).
- **One Dijkstra per origin point.** 222 Dijkstras for the example ROI; for a national run with a 1 km grid, roughly 1.1 million origins, each requiring a full Dijkstra over its window. This is the dominant cost and the reason the windowed design exists (§11.1). A multi-target Dijkstra that terminates when the cheapest destination is found, or a reversed Dijkstra from the road network (which answers “which origin points are closest to me?” for all of them at once), would be orders of magnitude faster.
- **The destination set is the rasterised road, not the road network.** Endpoint selection is “cheapest rasterised road cell within the window”, so the destination is quantised to 50 m and may be a road cell 50 m from the actual carriageway. The 50 m grid is a tunable that trades accuracy for speed and is currently hard-coded to a single value used for all of South Africa.
- **No treatment of the existing road as a corridor.** Roads are the destinations here, but where two origin cells route to the same road they will often duplicate long shared final legs, because the model has no notion of a shared spine or of marginal cost.
- **No uncertainty or confidence is propagated.** A route through terrain where the slope field is dominated by DEM noise is returned with exactly the same status as one through a well-defined quartz ridge. Given that the whole basis is a thresholded derivative of a noisy 30 m DEM, this is the largest single gap in the scientific presentation of the product.

3.4 Limitations of the endpoint definitions

- **Origin points are 1 km BSU cell centroids, snapped to the ROI.** The centroid is a point; the cell is 1 km². For an access road, the relevant thing is the distance from anywhere in the cell to the network, which is systematically less than the distance from its centroid. The model therefore over-states access distance on the order of half a cell, non-uniformly, and more so for cells whose road access is at one corner.

- **The BSU grid is administrative, not ecological or morphological.** 1 km cells do not align with veld, drainage or vegetation units, and a cell straddling a ridge and a valley is treated as one homogeneous unit.
- **The ROI clip is applied to points, not to cells.** 222 of 238 points survive ([clip_point](#), tests:94), i.e. the ROI boundary cuts the origin set at cell centroids. Some of the surviving cells are mostly outside the ROI and some excluded cells are mostly inside.
- Destination points are the centroids of rasterised road cells, so the finest achievable endpoint precision is the target raster resolution (50 m here, 100 m for the national product).

3.5 Limitations of the variable-window scheme

The windowing is the right idea and the edge-effect reasoning is sound, but there are three real issues.

- **Edge effects are mitigated, not eliminated, and never measured.** A 1 km buffer (nominally 0.01 degrees) around the window is a heuristic. There is no test that re-runs a subset with a much larger window and checks the routes are unchanged. This should be a validation gate (§5.9 gate 3), not an assumption.
- **The buffer is not symmetric.** B12 shows the sign error means the southern edge gets +0.01 and the other three get ± 0.01 , so the effective buffer ranges from 0 to 2 km depending on the edge. Verified against the committed [temp/extent.shp](#).
- **The window is chosen by a spatial nearest-neighbour heuristic, not by a computational budget.** For origin points in dense road-adjacent areas the window is small; for isolated points it is large. That is reasonable, but it means the compute cost per origin varies by orders of magnitude and the run time cannot be predicted. A budget-driven window (grow until either a target size or a path is found) would give predictable cost and a natural termination condition.
- **Origin points that fall on the window edge are placed on the zero-cost border ring (B04).** Verified for origin #0 in the committed example, which maps to row 0. Routes that begin there have a free corridor along the whole window boundary, which will bias the chosen endpoint toward the boundary.

3.6 The limitations, ranked by how much they affect the answer

Rank	Limitation	Direction of the error	Fixable by
1	No distance term — the objective is total climb only	Systematically over-long routes; wildly over-estimates access distance in gentle country	§5.3
2	Border ring and DEM voids are zero-cost corridors (B04)	Routes preferentially hug tile seams; catastrophic at national scale	One-line fix, §2 B04
3	Barrier semantics for water — no bridges or fords	Severe detours around every crossable river	§5.6
4	Hard 15 degree cut on unconditioned SRTM slope	Knife-edge feasibility; noise-driven spurious impassability	§5.2 and §5.7
5	Geographic-CRS routing, 13 % anisotropic	Systematic north–south bias; no distance in metres	§5.1
6	No land-cover / traction term	Cannot distinguish farm track from boulder field	§5.4
7	No turning cost, so switchbacks are free	Routes that a real tractor-trailer cannot follow	§5.5
8	Slope unit undocumented and not reproducible (B06)	The headline parameter cannot be stated or defended	§2 B06, §11.2
9	No uncertainty propagation	Over-confident results in exactly the regions where the DEM is worst	§5.9 and §10.3
10	Centroid-only origin definition	Systematic over-estimate of access distance, non-uniform	§5.1 (buffer the cells, not the points)

4 Variables that need to be parametrised

4.1 The complete inventory of hard-coded values

Every value below is a literal in the source. None of them is documented, none has a unit, and several have a unit that is different from what the comment claims. The recommended name encodes the unit, which makes unit errors impossible.

Parameter	Current value	Where	What it actually is	Recommended name and default
slope_threshold	15	tests:47	Threshold on a raster whose unit is undocumented (B06). The comment cites New Zealand's 10 degree heavy-vehicle limit and assumes a tractor-trailer, which is 15.2 % above it.	slope_threshold_deg = 12.0 (see §6.3)
na	-999999	tests:46, functions:1115, 1131	The NoData sentinel. Used both as a raster NoData value and as a NumPy barrier marker. The value matters: it must be negative for scikit-image to treat it as a barrier.	nodata = -1e9, with an explicit barrier array of dtype bool rather than a magic number
numpoints_roads	250	tests:113	Candidate destination points selected per origin point, purely to size the window.	n_candidates_per_origin = 250, auto-tuned down to min(250, n_road_points_in_window)
numpoints_window2	11	tests:115	Origin points per window. Drives both memory and the risk of the B08 index error.	n_origins_per_window = 11 (or better, computed from a memory budget)
window buffer	0.01	functions:1044	Degrees , commented as “1km buffer”. Inverted on the south side by B12.	window_buffer_m = 1 000, applied in the projected CRS
drain buffer	0.002	functions:1058	Degrees ≈ 20 m. Used to remove processed origins from the pool.	drain_radius_m = 20
for i in range(0, 24)	24	tests:120	Outer loop count. Should be $^{(222/11)} = 21$.	Remove entirely — loop until the pool is empty
BSU res	0.5	tests:82	The gdal.Translate resampling factor. 0.5 means “double the resolution” of the 100 m BSU, giving 50 m.	target_res_factor = 0.5, or just road_point_res_m = 50
BSU res2	50	tests:82	The geotransform resolution in metres. Redundant with res (B24).	road_point_res_m = 50 — one parameter, not two
scale	111120	functions:30	Passed to DEMProcessing as if it were a gdalDEM option. It is not one (B06).	dem_vertical_scale = 1.0, and use -p so the unit is degrees
fully_connected	True	functions:1157	8-connectivity. Not exposed, and not recorded in any output.	connectivity = 8, recorded in the manifest
analysis CRS	implicit EPSG:4326	throughout	Never named. Derived from whichever input happened to be passed first.	analysis_crs = the SANBI Albers Equal Area (see §6.1) — asserted at startup
routing CRS	EPSG:4326 (the road layer)	tests:90–91	Taken from NGI_road_path by reprojectshape_point.	analysis_crs — one CRS for everything
target point resolution	50 m (implicit)	tests:82	Precision of the road endpoint, and it multiplies the number of Dijkstra destinations.	road_point_res_m = 50 (national) / 100 (coarse)
obstacle burn value	0	functions:114, 216, 1123, 1140	A burn value of 0 combined with a NoData-initialised destination band (B23).	obstacle_burn_value = 1, with an explicit init_values
obstacle output prefix	"outputs/"	functions:104	Hard-coded relative directory, and the file is deleted immediately afterwards (B22).	Remove — use a MEM dataset
temp rasters	"temp/..."	functions:1114, 1134	Hard-coded paths inside the library, with _test in the names.	work_dir from the config
output prefix	"00000"	functions:1185–1188	The literal num="00000" is a vestigial attempt at zero-padding; n is then appended.	A run id and a stable origin_id; drop the padding
field width	50	functions:1207	OFTRail in a shapefile is capped at 20 bytes; 50 is silently clamped (B25).	Remove — move to GeoPackage, or drop the width
sys.maxsize	as infinity	functions:1166	An int sentinel in a float comparison.	np.inf
math.ceil() calls	redundant	functions:665, 669	math.ceil(len(x1)) and math.ceil(numpoints) on integers are no-ops.	Remove
2 036 / 515 / 505	—	verified in §2 B04	Not a parameter, but a number worth recording: the number of zero-cost border cells in the example ROI, i.e. 0.78 % of the	Remove the border — erode the crop by one cell

Parameter	Current value	Where	What it actually is	Recommended name and default
			raster.	

4.2 Parameters that do not exist yet but should

These are not currently present in any form. Each is required to make the model defensible or to make it usable for anything other than the single Uppington example.

Parameter	Purpose	Sensible default
cost_model	Which cost function to use (slope_only reproduces the current behaviour; traversal_time is the proposed one). Lets old results be reproduced after the model changes.	slope_only for a re-run, traversal_time going forward
w_drag, w_climb, w_steep, w_traction, w_turn	The coefficients of the proposed cost function (§5.8).	calibrated, not guessed — see §5.9
theta_soft_deg, theta_hard_deg	The two slope breakpoints of the graded steepness penalty, replacing the single hard cut.	8 and 18, by terrain zone — §6.3
slope_penalty_beta	The exponent that makes the steepness penalty super-exponential above theta_soft.	3 to 5; chosen by the convergence test
obstacle_penalty	A large finite cost for water instead of an absolute barrier, plus a per-crossing bridge or ford cost.	100× the median cost of a compliant 1 km route
surface_raster	Path to a land-cover raster used for the traction term (§5.4).	SA land-cover raster; absence disables the term
allow_crossings / crossing_cost	Whether river crossings are permitted and what they cost.	False reproduces the current behaviour
dem_condition	A boolean controlling void filling, despiking and smoothing before differentiating.	True
min_route_length_m	Routes shorter than this are classified as already_served rather than reported. The committed example has 14 routes under 200 m, the shortest 27 m.	50 m (half the target resolution)
confidence_threshold	The route-length / cost stability tolerance that a route must meet to be reported as high confidence.	5 % between the two window sizes
overwrite / run_id	Whether to overwrite an existing output, and the identifier stamped on every record.	overwrite = False, run_id = UTC timestamp
seed	Only needed if any stochastic step is added (e.g. a randomised tie-break).	42

4.3 A single configuration object

```
# configs/uppington_example.yaml
run_id:          null          # generated if null

crs:
  analysis:      EPSG:2054      # SA Albers Equal Area, or a named .prj path
  input_roads:   EPSG:4326
  output:        EPSG:4326

paths:
  dem:           inputs/SRTM_clip.tif
  bsu:           inputs/BSU_100m_clip.tif
  roads:         inputs/roads/NGI_2019_road_clip.shp
  obstacles:     inputs/obstacles/
  roi:           inputs/ROI.shp
  work_dir:      work/
  out_dir:       out/

grid:
  origin_resolution_m: 1000      # the BSU aggregate that defines origin points
  road_point_res_m:    50

cost:
  model:         slope_only
  slope_threshold_deg: 12.0
```

```
dem_condition:    true
obstacle_penalty: 1.0e6

window:
  n_origins:      11
  buffer_m:       1000
  n_candidates_per_origin: 250
  drain_radius_m: 20
  max_cells:      400000      # memory guard, replaces the hard-coded 24

routing:
  connectivity:    8
  allow_crossings: false
  min_route_length_m: 50

output:
  format:          gpkg
  crs:             EPSG:4326
  overwrite:       false
```

5 Improvements to the physics and the equations

The following are ordered by value delivered per unit of effort. Items 5.1 and 5.2 alone would change the product qualitatively; the rest make it defensible.

5.1 Do the routing in a projected, equal-area, isotropic CRS

This is the single highest-value change and it is nearly free. The SANBI / StatsSA BSU raster the model already uses is in Albers Equal Area with metre units. Routing in that CRS instead of EPSG:4326 fixes four separate problems at once:

- **d_uv is in metres.** The MCP edge weight becomes a real distance, so a cost of 1 means one metre, and `dist_m` becomes computable by a simple sum.
- **The grid is isotropic.** Equal-area conic projections have very small scale distortion over a single country's extent. The 13 % N–S/E–W anisotropy that biases routing north–south disappears.
- **Buffers are in metres.** `window_buffer_m` and `drain_radius_m` become literal, and the latitude-dependence and the B12 sign error both become impossible.
- **Nearest-neighbour selection becomes isotropic.** `euclidean_distances` on metre coordinates is now a proper Euclidean nearest-neighbour search, and the file's unused haversine helper is no longer needed.
- **Area comparisons become meaningful.** Useful if origin cells are ever compared by area rather than by centroid distance.

Two options. Either (a) reproject the cost raster into the Albers grid before routing and keep point layers in the same CRS throughout — simplest, and the DEM resampling is benign because a resampling of a slope surface to a 27×31 m metric grid changes very little. Or (b) keep the slope in EPSG:4326 and only pass `spacing` to `MCP_Geometric` — better if the installed scikit-image supports it (a `spacing / sampling` argument was added in the 0.24–0.25 line), but it fixes only the anisotropy, not the units, the buffers or the distances. **Option (a) is recommended.**

If the routing CRS is changed, the output CRS should stay EPSG:4326 (it does today) so the product is unchanged for consumers — but the reprojection must now be explicit and recorded, not incidental.

5.2 Replace the hard slope cut with a graded, super-exponential penalty

The current `slope[slope >= 15] = na` is a step function. Replace it with a continuous penalty that is near zero below a soft threshold and rises steeply above it:

```
def slope_penalty(theta_deg, theta_soft, theta_hard, beta, w_steep):
    """theta_soft: free. theta_hard: prohibitive. beta: how fast it blows up."""
    excess = np.maximum(0.0, theta_deg - theta_soft)
    span = max(theta_hard - theta_soft, 1e-6)
    P = w_steep * (1.0 + excess / span) ** beta          # P(theta_soft) = w_steep
    P = np.where(theta_deg >= theta_hard, P * 1e6, P)    # effectively impassable
    return P
```

Why a power law rather than an exponential or a logistic: it is monotone, it is zero below `theta_soft` so the flat, easily-driven country is not penalised at all, and the exponent `beta` is a single interpretable knob ($\beta = 1$ is linear, $\beta = 3$ – 5 is steeply super-linear). A logistic saturates, which would make a 40 degree slope no harder than a 25 degree one — undesirable. `theta_hard` preserves the model's ability to declare genuinely impassable ground without relying on a NoData sentinel, and it makes the threshold visible and tunable as two numbers rather than one. Note that with the current model the absolute value of the threshold does not change the routing at all (see §3.1); with a graded penalty the value of `theta_soft` and the ratio of the two weights now genuinely control the answer, which is a much more honest situation.

5.3 Add a distance term so the objective is not 'least climb'

Without a distance term the model will happily add tens of kilometres to avoid a small rise. The cheapest possible fix is to add a constant per-metre cost alongside the slope term. Conceptually, the cost per step becomes:

```
C(x, y) = w_dist * 1.0                # per metre travelled
          + w_steep * slope_penalty(theta)    # graded steepness
          + w_climb * max(0, tan(radians(theta))) # climb energy, Tobler-like
```

$$\text{edge weight} = 0.5 * (C_u + C_v) * d_{uv} \quad \text{with } d_{uv} \text{ in METRES}$$

The balance between `w_dist` and `w_steep` is the key calibration. A physically defensible anchor is the energy argument: for a vehicle of mass m , the gravitational work to climb Δh is $m \cdot g \cdot \Delta h$, while the rolling and aerodynamic work over distance L is roughly $(c_{rr} \cdot m \cdot g + \frac{1}{2} \rho \cdot C_d \cdot A \cdot v^2) \cdot L$. Setting them equal gives a crossover grade independent of vehicle mass, which is the standard result and a good starting point for a logistics model. Everything else in the cost function can then be expressed relative to that.

5.4 Add a traction term from land cover

Reclassify a land-cover raster into a small number of traction classes with a coefficient of rolling resistance, and add `w_traction /  $\mu(x, y)$` to the per-metre cost. A workable South African classification:

Class	Typical cover	μ (coefficient of rolling resistance)	Comment
Compacted gravel / sand track	Gravel, sand, old roadbed	0.02–0.03	Baseline
Firm soil / short grass	Natural veld, crop residue	0.05–0.08	Usable when dry
Long grass / bush	Thicket, shrubveld	0.10–0.15	Seasonal; see §6.6
Rough / stony	Rock outcrop, scree, erosion gully	0.20–0.30	Often impassable when wet
Sand / soft sand	Kalahari deep sand, dunes	0.15–0.40	Highly depth-dependent
Wetland / water	Vlei, pan, river	impassable	The current barrier, correctly
Cultivated land	Maize, groundnuts	0.05–0.10	Seasonal; access consent issues

This single term is what would let the model distinguish a farm track across the Highveld from a route through the Soutpansberg, and it is also what makes the result seasonable (§6.6). If no land-cover raster is available, the term should be absent rather than defaulted to a constant — and the absence recorded in the manifest.

5.5 Add a turning / switchback penalty

A tractor-trailer combination of 20–25 m cannot turn on a 30 m grid step. The model currently treats a path that reverses direction every three pixels as free. A simple and effective correction is to add a cost to every step whose heading differs from the previous step's heading by more than a threshold:

```
turn_penalty( $\Delta$ heading) = w_turn if | $\Delta$ heading| > turn_threshold_deg else 0

# 8-connected headings: 0, 45, 90, 135 degrees. A 180-degree reversal is the expensive case.
# turn_threshold_deg = 25 keeps 0 and 45 degree continuations free.
```

This is not expressible in `MCP_Geometric` as shipped, because its cost model is memoryless — the edge cost depends only on the two endpoint cells. It requires either a custom Dijkstra over a state-augmented graph (state = previous heading, which multiplies the node count by 8) or a post-processing pass that penalises reversals. **Recommendation:** do the post-processing pass first (measure how many of the existing routes actually reverse, and by how much), and only justify the state-augmented graph if the answer is “most of them”.

5.6 Make river crossings representable

Replace the absolute barrier with a large finite cost, and add a separate crossing cost that is charged once per water cell entered. Three parameters: `obstacle_penalty` (the cost of being in a water cell at all), `allow_crossings` (a boolean), and `crossing_cost` (what a bridge or ford costs, in the same units as the rest of the cost). With `allow_crossings: false` this reproduces the current behaviour exactly, which makes it a safe, opt-in change. It is also the change most likely to alter results materially, because the Orange, Vaal and Tugela currently act as continent-scale walls.

5.7 Condition the DEM before differentiating it

The slope of unconditioned elevation data is dominated by the noise in the noise. Three cheap, well-established steps, all available in GDAL or SciPy:

- **Void filling.** Iterative fill of the NoData region from the surrounding valid cells, so that voids become flat (slope 0) rather than becoming barriers or, under B04, free highways. `scipy.ndimage.distance_transform_edt` with `return_indices=True` gives a nearest-neighbour fill; or GDAL's `gdal_fillnodata.py`.
- **Despiking.** SRTM v3 has isolated one- and two-pixel spikes. A 3×3 median filter, or a conditional filter that replaces a cell only when it differs from the median of its neighbours by more than several times the local MAD, removes them without touching real terrain.
- **Smoothing.** A light Gaussian (sigma well under one cell) reduces the high-frequency content that dominates the derivative. One to two cells of smoothing is defensible at 30 m; more than that starts to bias the gradient in genuinely broken country such as the Drakensberg escarpment.

Also worth doing, and cheap: a topographic position index (TPI) or roughness layer as a **tie-break** or as an additional mild cost term. A local roughness measure distinguishes ‘gentle and smooth’ from ‘gentle and chaotic’, which is exactly the distinction the current model cannot make. It also provides a natural proxy for DEM uncertainty, which feeds §5.9.

5.8 The proposed cost function, written out

$C(x, y)$	$= w_{\text{drag}} \cdot (L / L_0)^{b_{\text{drag}}}$	rolling + aerodynamic drag, per metre
	$+ w_{\text{climb}} \cdot \max(0, \text{grade})^{b_{\text{climb}}} \cdot L$	climb energy, Tobler-like
	$+ w_{\text{steep}} \cdot P(\theta) \cdot L$	graded steepness penalty (§5.2)
	$+ w_{\text{trac}} \cdot (1 / \mu(x, y)) \cdot L$	traction, from land cover (§5.4)
	$+ \sum w_{\text{turn}} \cdot 1[\Delta \text{heading} > \theta_{\text{turn}}]$	switchbacks (§5.5)
	$+ \text{crossing_cost}(x, y)$	finite, parameterised (§5.6)

$\text{edge weight}(u, v) = \frac{1}{2} \cdot (C(u) + C(v)) \cdot d_{uv}$, d_{uv} in METRES

reported per route:

<code>gross_length_m</code>	$\sum d_{uv}$
<code>net_climb_m</code>	$\sum \max(0, \Delta z)$ <code>descent_m</code> = $\sum \max(0, -\Delta z)$
<code>mean_grade_pct</code>	$100 \cdot \text{net_climb_m} / \text{gross_length_m}$
<code>max_grade_deg</code>	max over steps of the local slope
<code>est_travel_time_h</code>	$\sum d_{uv} / v(\text{grade}, \mu)$ [only if a speed model is added]
<code>n_crossings</code>	count of water cells entered
<code>surface_cover_mix</code>	length per land-cover class
<code>already_served</code>	TRUE if the origin cell is itself a road cell
<code>confidence</code>	from the convergence test (§5.9)

Every coefficient is optional. A configuration with only `w_drag` and `w_steep` set is already a large improvement on the current model, and a configuration with all of them set is a genuinely defensible logistics surface. This is the practical advantage of the decomposition: the model can be adopted incrementally.

Figure 6 — Turning a slope integral into a defensible traversal-time surface

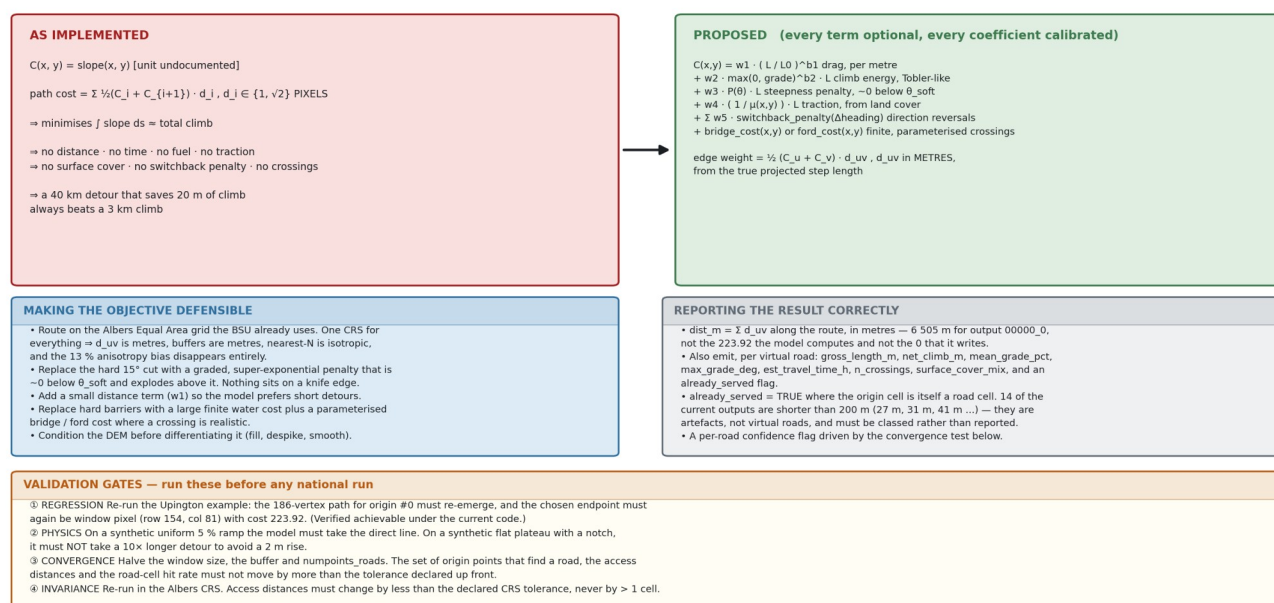


Figure 3 — The proposed decomposition of the cost function, the reporting set that goes with it, and the four validation gates that must pass before a national run.

5.9 Calibration, and the four validation gates

The coefficients in §5.8 must be calibrated, not chosen. A workable procedure, in order:

- **Start from physics, not from fitting.** Set the drag-to-climb ratio from the energy crossover grade (§5.3) and the rolling-resistance coefficients from a standard table for the land-cover class. This gives a defensible starting point with no data required.
- **Then fit the two free exponents and the steepness weight.** Against a set of surveyed agricultural access routes: take 30–50 real farm roads / firebreaks / pipeline routes with known endpoints and geometry, and choose b_{climb} , β and w_{steep} so the model reproduces them. Twenty to thirty routes spread across the veld types of §6.3 would be enough to constrain three parameters, and the routes double as an independent validation set if you fit on half and test on the other half.
- Then run the four gates below on every candidate configuration, and keep the configuration that passes all four.

Gate	What it tests	Pass criterion
1 Regression	The committed Upton example, under cost_model: slope_only.	The 186-vertex route for origin #0 re-emerges; the chosen endpoint is window pixel (row 154, col 81); the cumulative cost is 223.92 to 4 significant figures. This proves the refactor did not change the algorithm.
2 Physics	Synthetic terrain with a known answer.	On a uniform 5 % ramp the route is the direct line. On a flat plateau with a 2 m notch, the route does not take a route 10× longer to avoid it. On a 5 % ramp with a single 30 m-wide barrier, the route either crosses (if allow_crossings) or detours along the barrier, and does something in between.
3 Convergence	The same ROI, with the window size, the buffer and n_candidates_per_origin all halved.	The set of origin points that find a road changes by at most the stated tolerance; access distances change by less than 5 %; no origin point flips between routed and no_path. This is the gate that proves the windowing is not creating the answer.
4 Invariance	The same ROI, routed in the Albers CRS instead of EPSG:4326.	Access distances change by less than one cell width. A larger change means the geographic-CRS anisotropy is driving the result, which is the failure mode §2 B07 and §5.1 exist to eliminate.

6 Parameters that need to be optimised for South Africa

6.1 Start with the right national CRS

This is the parameter with the largest effect on the whole national product, and it is not a number. Three candidates, in order of recommendation:

Option	Definition	Assessment
Albers Equal Area (the BSU's own CRS) — recommended	<code>+proj=aea +lat_1=-22 +lat_2=-38 +lat_0=-30 +lon_0=25 +x_0=1400000 +y_0=1300000 +datum=WGS84 +units=m</code>	Equal-area, so areas — and therefore the 1 km origin cells — are true. Scale distortion across South Africa's 29 degrees of latitude is a few percent. It is the CRS the BSU grid is already in, so no new parameter is needed and the origin points need no conversion. This is the natural home for the whole analysis.
Hartebeesthoek 94 / Lo19 (EPSG:2054 family)	A conformal conic optimised for South Africa.	Conformal, so shapes and angles are right but areas are not. Better than geographic for distance and bearings, and it is the legal / cadastral CRS of the country. Reasonable choice if you need a single well-known EPSG code, but the BSU would still need reprojecting and the 1 km cells would not be exactly 1 km ² .
EPSG:4326 — the current state	Geographic degrees.	Anisotropic (13 % at the Upington latitude, worse at the Cape), non-metric, and it makes every buffer and every distance a latitude-dependent quantity. Not defensible for a routing model. Retain it only as the *output* CRS if downstream consumers expect it.

A sensible arrangement: run everything in Albers, write the product in both Albers (for analysis) and EPSG:4326 (for interoperability with the NGI and StatsSA layers consumers already use). The output CRS is then an explicit, recorded choice rather than an accident of input order.

6.2 The DEM, and where the model is weakest

SRTM v3 at 1 arc-second is a reasonable national choice: free, complete over South Africa, and with a well-known error envelope. But its vertical error is not uniform and the terrain is not either. The table below is a qualitative reliability map for the routing, based on the interaction of vertical error and local relief.

Region	Relief over 30 m	SRTM v3 behaviour	Routing reliability	Mitigation
Highveld / Witwatersrand (Gauteng, Mpumalanga, North West)	Low to moderate, 50–150 m per km	Usable. Some C-band stripe artefacts on the eastern Highveld.	Moderate to good	Despike; consider a 5 m LiDAR DEM where available (City of Johannesburg, eThekweni, Tshwane)
Drakensberg / Great Escarpment (KwaZulu-Natal, Free State, Lesotho border)	Extreme, 200–500 m per km	Good relative geometry but genuine voids on steep shadowed slopes; more C-band noise.	Good for the steepness mask, poor for absolute route length	Fill voids; the steepness threshold will remove most of the affected area anyway
Kalahari / Northern Cape (Upington, Kimberley, Griqualand West)	Very low, 5–30 m per km	Worst case. 200 m of total relief over 15 km means the terrain signal is comparable to the 5–9 m vertical error. Slope is substantially noise.	Low — most of the committed example ROI is in this class	Smooth aggressively; add a roughness term; report a low confidence flag; consider a 10–20 m commercial DEM (AeroSouth, Airbus Pléiades)
Karoo (Western and Eastern Cape interior)	Low to moderate	Generally clean.	Moderate	Despike; calibration against real access routes (\$5.9)
Limpopo / Mpumalanga lowveld (bushveld)	Low, with local breaks	Clean, but dense vegetation is not represented at all	Moderate	The land-cover traction term of \$5.4 matters most here
Cape Fold Belt (Western Cape, southern Cape)	Sharp ridges, deeply incised valleys	Good in the valleys, noisy on the ridge crests	Moderate to good	Despike; expect the steepness mask to be the binding constraint, which is correct
Namib / Atlantic coastal strip	Sharp, arid, deep sand	Clean.	Moderate	The deep-sand traction class (\$5.4) is essential here
Sabi–Limpopo and Kruger lowveld	Low	Clean	Moderate	Protected-area constraints (\$6.8) become the binding constraint, not terrain

Practical recommendation: produce the national product in **relief zones**, not as one run with one parameter set. The nine regions above naturally group into three or four calibration zones, each with its own `theta_soft`, traction weights

and smoothing strength. Reporting a single national number for access distance implicitly assumes the model is equally reliable in the Kalahari and on the Drakensberg, and it is not.

6.3 Slope thresholds by veld type

The current single threshold of 15 degrees is cited in the code comment as being for a tractor-trailer combination, derived from New Zealand's 10 degree heavy-vehicle limit plus a 50 % margin. That is a reasonable first guess but it ignores the two things that actually govern traversability: the surface (see §5.4) and the vehicle. A fully loaded sugarcane tractor-trailer or a 30 tonne biomass truck on a wet clay road is a very different problem from an empty tractor on a dry gravel farm road.

Veld / terrain zone	Recommended `theta_soft_deg`	Recommended `theta_hard_deg`	Rationale and additional considerations
Kalahari and Nama sandveld (Northern Cape, Free State, North West)	6	14	Flat, deep sand. The binding constraint is traction and moisture, not gradient — a very low <code>theta_soft</code> with a strong sand traction weight is correct. Add: summer versus winter trafficability.
Karoo and succulent Karoo (Western, Eastern, Northern Cape)	9	18	Gravelly, stony, generally firm. Moderate thresholds; local rock outcrop will need to be identified separately from gradient.
Hesveld / Veld (Gauteng, Mpumalanga, North West, Limpopo)	8	16	Firm soils, cultivation, good road networks. The road-access question is usually not the binding one here.
Bushveld / Lowveld (Limpopo, Mpumalanga, KwaZulu-Natal)	7	15	Dense vegetation and seasonal wetness. Traction and season dominate.
Montane grassland (Free State, KwaZulu-Natal, Eastern Cape)	10	20	Good traction, moderate slopes, but frost and wet-season degradation are real. Winter access is materially worse.
Drakensberg / Afro-montane (KwaZulu-Natal, Free State, Eastern Cape)	12	22	Highest accessible thresholds. Steep but often well-drained and firm. Note that the Griqualand and Drakensberg slope-aspect (aspect matters enormously, and is not modelled) should eventually be added.
Afro-montane forest belt (Limpopo, Mpumalanga, KwaZulu-Natal)	8	16	Gradient is rarely the binding constraint; biomass density and legal access are. Consider a lower <code>w_steep</code> and a much higher <code>w_trac</code> .

These are starting values for calibration, not results. The procedure is §5.9: fit against surveyed access routes in each zone, then report the fitted values with the fit statistics. The single most valuable addition to this table is the **standard error on each threshold**, which requires real routes and which is currently entirely absent.

6.4 Vehicle and load parameters

The README describes the vehicle as a “tractor trailer combination” for biomass logistics. The mass, dimensions and loading determine the tractive limits and must be parameters, not assumptions.

Parameter	Why it matters	How to use it
<code>vehicle_mass_kg</code>	Gravitational work per metre of climb, $m \cdot g \cdot L \cdot \text{grade}$. Sets the crossover grade against rolling resistance.	Enters <code>w_climb</code> ; cancels out of the drag-to-climb ratio, so it mostly matters for absolute energy rather than routing.
<code>vehicle_length_m</code> (20–25 for a road train)	Minimum turning radius. A 22 m combination cannot follow a 30 m radius hairpin.	Drives <code>theta_turn</code> and the minimum radius that the switchback penalty of §5.5 permits.
<code>vehicle_width_m</code> (2.5–3.0)	Single-lane width. Two-lane passing is a different product.	Feeds the road-class weighting in §6.5 and the minimum corridor width.
<code>axle_load_max_kg</code> (South African legal limits are 8000 kg on a single axle and 18 000 kg gross for a 6-axle rigid; 22 000–24 000 kg for a well-designed combination)	Determines which road classes and surfaces are usable at all, and where bridges and causeways are permitted.	Selects the road classes in §6.5 and gates bridge/ford crossings (§5.6).
<code>load_state</code> (empty / full / part)	A full vehicle's tractive limit is far lower than an empty one's.	Run the model for the empty case and scale by a load factor, or report a range.
<code>operating_speed_kmh</code> by grade and surface	Converts cost to travel time.	Only needed if a time product is wanted; then use a graded speed model rather than a constant.
<code>fuel_consumption_l_per_km</code> by grade	Converts cost to cost-per-hectare.	The quantity a biomass-logistics feasibility study actually wants.

6.5 The road network: which roads count as a destination

The NGI road layer has a class attribute that the model ignores entirely — it rasterises the whole layer. That conflates a national tarred road with a cattle track, and it means a virtual road may be routed to a destination the vehicle cannot legally or practically use.

NGI road class (approximate)	Vehicle that can use it	Suggested treatment
Primary arterial (N1, N2, ...)	Any	Destination, weight 1.0
Secondary / regional (R road)	Any heavy vehicle	Destination, weight 1.0
Local / distributor (numbered)	Most	Destination, weight 1.0; the default
Public road, gravel	Heavy vehicles	Destination, weight 0.8 — and note that these degrade in the wet season
Other / farm road / track	Only light vehicles in many cases	Destination only if <code>allow_light_roads</code> , else excluded — but the 1 km BSU origin grid is often only reachable via exactly these
Footpath / cycle track / bridle path	Not a road vehicle	Exclude by default; make it a flag, because excluding them changes the answer materially in some districts

Note the tension to resolve explicitly: if the model excludes light tracks, many 1 km cells become “unreachable” and the product becomes a statement about the classified road network rather than about biomass accessibility. That may be the right product, but it should be a decision with a documented flag, not a side effect of rasterising everything.

6.6 Seasonality and climate — currently entirely absent

South African agricultural access is strongly seasonal and the model has no way to express it. This is the largest single omission if the intended use is biomass logistics.

- **Rainfall seasonality by zone.** The summer-rainfall interior (Highveld, Mpumalanga, Limpopo, northern KwaZulu-Natal) has a November–March wet season during which unpaved roads are frequently impassable to heavy vehicles. The winter-rainfall Cape is the opposite. The model can produce a `season` parameter and be run per season, or a single-year mean with a wet-season worst case.
- **Drought.** The 2015–2019 and 2022–2024 droughts pushed farmers onto tracks that are not passable in a normal year, and pushed the practical network further from the classified network. A drought-year flag is a legitimate scenario to publish alongside the baseline.
- **Frost.** Montane grassland frosts can make a soft track rutted and effectively closed for weeks.
- **Burning and vegetation.** Prescribed burns in the veld change the traction class of the same track within a single season; this is a land-cover refresh frequency issue, not a model issue.
- **Load-shedding and fuel.** Not a terrain parameter, but if the product is meant to support routing decisions, expected fuel cost per route (from the cost model of §5.3) is far more actionable than distance.

6.7 Other South-African factors that should be explicit flags

Factor	Why it belongs in the model	Implementation
Protected areas and conservation areas (SANBI layer)	A route through a protected area is a legal and social question, not a terrain question. For biomass supply chains this is often decisive.	Add as a very large finite penalty or a hard mask, with an <code>allow_protected</code> flag and a separate count of how many routes are affected.
Communal land and tenure	Access requires negotiation; the cost is social, not physical.	At minimum, report the tenure composition of each route. <code>surface_cover_mix</code> generalises to <code>tenure_mix</code> .
Existing private farm roads and firebreaks	These are the network a farmer would actually use and they are not in the NGI layer. This is a data gap, not a model gap — but the model should make it possible to add them as additional destination layers with a lower cost weight.	
Bridges and culverts (SANIX, NBI)	Needed for §5.6 to be usable rather than theoretical.	A point layer that reduces <code>crossing_cost</code> to near zero at existing structures.
Vleis and wetlands (NEMA)	Ecologically and legally constrained, often permanently	Separate from rivers; treat as a hard mask with a flag.

Factor	Why it belongs in the model	Implementation
	wet.	
Sand mining and quarry sites	Common in the Limpopo and KZN; block access and are a hazard.	Point/area obstacles.
Load and axle limits per bridge	Determines which river crossings are possible at all.	Join the bridge layer's capacity to axle_load_max_kg.

6.8 National scale: resolution and volume

The README records a merged national cost surface of 2 711 921 602 pixels. The windowing scheme exists to make that tractable, and the parameters that determine whether it is tractable are `n_origins_per_window`, `window_buffer_m` and `n_candidates_per_origin` — all currently hard-coded and all with no memory guard.

Choice	Current / documented	Assessment
Origin grid	1 km BSU aggregate (100 m for the example ROI's source grid is 1 km after aggregation)	Appropriate. Coarser (2 or 5 km) reduces the product's resolution; finer (500 m) roughly quadruples the run time for little gain.
Road point resolution	50 m in the example; 100 m for the national product per the README	100 m for the national run is the right compromise and should be the documented default, not an accident of the example. It also directly quantises the destination.
DEM resolution	1 arc-second (≈ 30 m)	Retain. A 90 m DEM would halve the cells but the steepness mask becomes meaningless in broken country; a 10 m DEM would be far better but is not publicly available nationally at acceptable cost.
Origins per window	11	Must be derived from a memory budget, not chosen. A safe rule: $n_origins = \text{floor}(\text{target_bytes} / \text{bytes_per_pixel} / \text{cells_per_window})$, with <code>target_bytes</code> around 4 GB, and the window cell count from the empirical relationship between window size and the number of buffered origin points.
Estimated national volume	about 1.1 million 1 km cells; 220 outputs for 222 origins in the example implies roughly one shapefile set per cell	At 880 files per 220 roads, a national run produces on the order of four million files in the current output structure. This is the single strongest practical argument for the GeoPackage change in §10.4 — a flat directory with millions of files is unusable on most file systems and in most GIS software.

6.9 A recommended South African default parameter set

A defensible starting configuration for a national run. Every value is a starting point for calibration (§5.9), not a result.

```
crs:
  analysis: SANBI Albers Equal Area (aea, lat_1=-22, lat_2=-38, lat_0=-30, lon_0=25,
    x_0=1400000, y_0=1300000) # see §6.1
  output: EPSG:4326

grid:
  origin_resolution_m: 1000 # the BSU aggregate
  road_point_res_m: 100 # national; 50 m for provincial work

cost:
  model: traversal_time
  slope_soft_deg: 9 # §6.3, single-value compromise; use per-zone in production
  slope_hard_deg: 18
  slope_penalty_beta: 4
  w_drag: calibrated # §5.9 -- the drag:climb ratio from the energy crossover grade
  w_climb: calibrated
  w_steep: calibrated
  dem_condition: true # fill + despoke + 1.5-cell Gaussian (§5.7)
  surface_raster: null # enables the traction term when supplied
  obstacle_penalty: 1.0e6 # effectively a barrier; §5.6 makes this finite and tunable

window:
  n_origins: auto # from a memory budget, not typed (§6.8)
  buffer_m: 1500 # 1.5x the expected access distance, not 1 km
  n_candidates_per_origin: 250
  drain_radius_m: 20
  max_cells: 400000
```

```

routing:
  connectivity:      8
  turn_threshold_deg: 25      # $5.5
  allow_crossings:   false    # opt in with a bridge layer ($5.6, $6.7)
  min_route_length_m: 50

output:
  format:            gpkg
  overwrite:         false
  confidence_threshold: 0.05

```

7 How the coordinate systems work

This section is a complete account of every coordinate operation in the code: what each CRS is, where it comes from, how the raster geotransform maps between world coordinates and array indices, what the four conversion functions do, where the conventions are right, and where they are wrong. It is written so that someone who has never opened the code can follow it, and it is precise enough that the arithmetic can be checked.

7.1 The three coordinate reference systems in play

The code touches exactly three distinct CRSs. Two are declared in the input data; one is implied by the data format.

CRS	Where it comes from	Units	Isotropic?	Used by
EPSG:4326 — WGS 84 geographic	inputs/SRTM_clip.tif (GeoTIFF CRS), the .prj of ROI.shp, NGI_2019_road_clip.shp, hydro_clip.shp, 1000m_source_wgs84.shp, road_50m_target_wgs84.shp, and every output .prj.	decimal degrees	No — 1.13 : 1 at −28.4 degrees	The DEM; the cost surface array that is actually routed on; all four point layers; every distance, buffer and window parameter; all outputs.
Albers Equal Area (the SANBI / StatsSA BSU native CRS)	inputs/BSU_100m_clip.tif (GeoTIFF CRS, EPSG:2054-style parameters); the .prj of 1000m_source.shp; the .prj of road_50m_target.shp (428 bytes — the full Albers WKT).	metres	Yes (equal-area conic, a few percent scale distortion nationally)	The definition of the origin points and the road points. Not used for routing.
SRTM v3 storage grid — not a CRS of its own	The source tiles are EPSG:4326, 1 arc-second, posted on the WGS 84 ellipsoid.	decimal degrees	No	The elevation data from which slope is derived. The distinction from EPSG:4326 matters because the ground sample *distance* is defined on the ellipsoid, not on the terrain: 27.2 m E–W × 30.7 m N–S at this latitude.

Figure 3 — The three coordinate systems, and the pixel ↔ world transforms

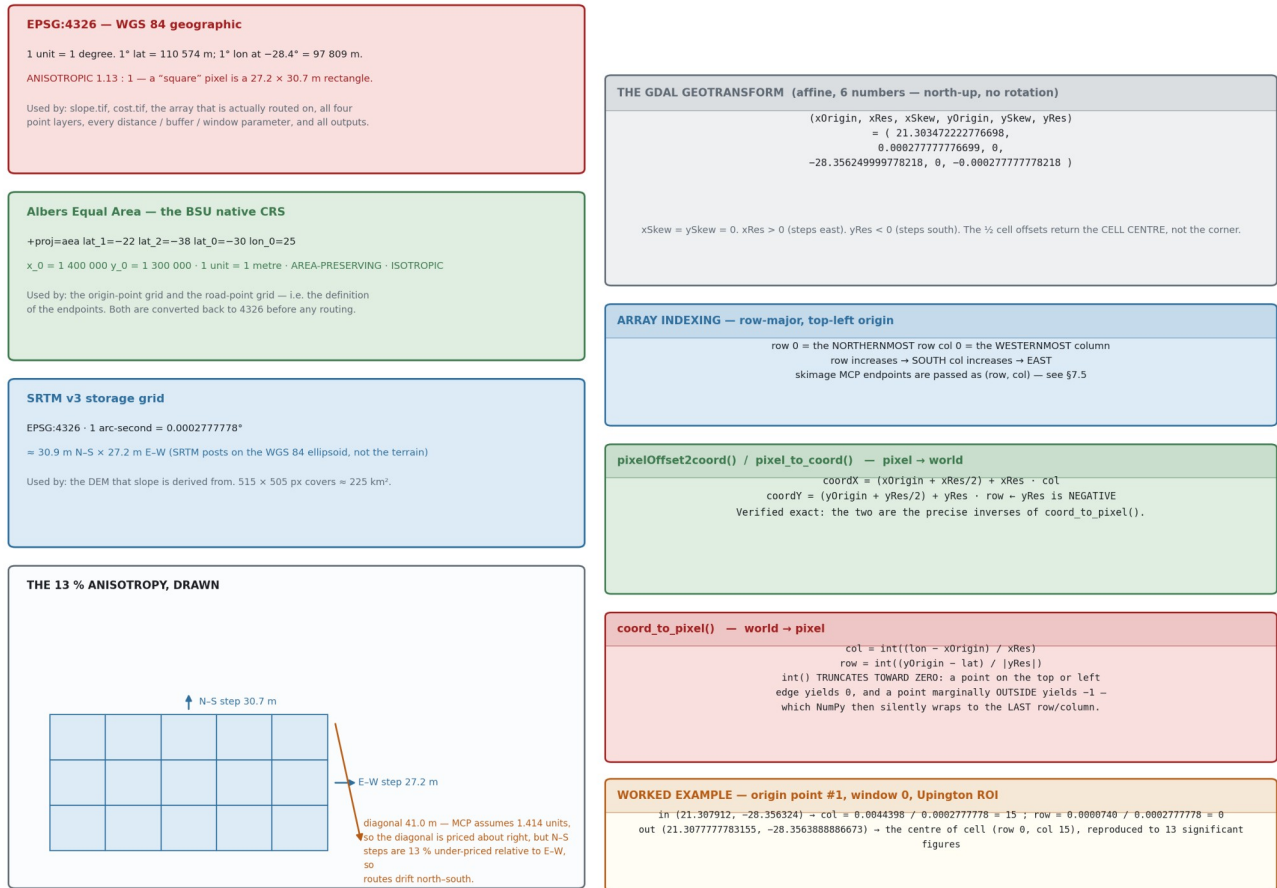


Figure 4 — The three coordinate systems, the geotransform, and the pixel ↔ world transforms. The arithmetic in this figure is correct; the CRS choice is not.

7.2 The geotransform

Every GDAL raster carries a six-number affine transform, and the code reads it correctly everywhere it is read. The transform for the ROI cost raster is:

```
(xOrigin, xRes, xSkew, yOrigin, ySkew, yRes)
= ( 21.303472222776698,
    0.000277777776699, 0,
    -28.356249999778218, 0, -0.000277777778218 )
```

and, with the two pixels written out:

```
x(pixel col) = 21.303472222776698 + col * 0.000277777776699
y(pixel row) = -28.356249999778218 + row * (-0.000277777778218)    # yRes is NEGATIVE
```

- **xOrigin, yOrigin** are the coordinates of the **upper-left corner** of the upper-left cell, not its centre.
- **xRes** is positive: increasing **col** steps **east**. **yRes** is negative: increasing **row** steps **south**. This sign convention is the single most common source of sign bugs in raster code, and this code gets it right in all four places it is used.
- **xSkew** and **ySkew** are 0, so the grid is north-up and unrotated. This is verified from the committed data, not assumed.
- **0.000277777776699** degrees is 1/3600 to within 1 part in 10⁸, i.e. 1 arc-second.
- Rasters also expose **x_min**, **x_max**, **y_min**, **y_max**. The code computes these in six functions (**x_max = x_min + ds.RasterXSize * xres** at functions:91, 155, 201, 556, 797 and tests:56) and then **never uses any of them**. They are dead code in all six places.

7.3 Array indexing: the (row, col) convention

A NumPy array of a raster is indexed `[row, col]`, row-major, and the convention in the code is consistent with the geotransform:

	Direction	Value at index 0	Note
row	increases southward	the northernmost row	row 0 is the top of the image
col	increases eastward	the westernmost column	col 0 is the left of the image
value	—	—	<code>array[row][col]</code> (numpy) is the same as <code>array[row, col]</code>

This is the correct and conventional mapping, and it is used consistently — which is worth saying because it is the most commonly botched part of raster routing code.

7.4 The four coordinate functions

Function	Location	Direction	Formula as coded	Assessment
<code>pixelOffset2coord()</code>	functions:308–317	pixel → world	<code>coordX = (originX + pixelWidth/2) + pixelWidth * xOffset</code> <code>coordY = (originY + pixelHeight/2) + pixelHeight * yOffset</code>	Correct. The % offsets return the cell centre rather than the corner, which is what a point layer should contain. Note it takes the offsets as (<code>xOffset</code> , <code>yOffset</code>) = (<code>col</code> , <code>row</code>) in that order, which is the opposite of the array-index order — a readability trap, not a bug.
<code>pixel_to_coord()</code>	functions:1275–1285	pixel → world	Byte-for-byte identical to the above.	Correct but duplicated (finding S7). Identical variable names, identical arithmetic. One of the two should go.
<code>coord_to_pixel()</code>	functions:1241–1255	world → pixel	<code>col = int((gridvaluesX[k] - xOrigin) / pixelWidth)</code> <code>row = int((yOrigin - gridvaluesY[k]) / pixelHeight)</code> with <code>pixelHeight = -geotransform[5]</code>	Arithmetically correct, but it truncates and it does not bound-check. Three consequences: (i) a point exactly on the top or left edge can yield 0 rather than being rejected; (ii) a point marginally *outside* the raster yields <code>-1</code> , and NumPy then silently wraps it to the last row or column; (iii) the caller is left to discover the problem, and the caller is a bare <code>except: pass</code> . It also returns <code>data[row][col]</code> , reading the whole raster a third time purely to fetch one value that the caller discards.
<code>coord_to_pixel2()</code>	functions:1257–1271	world → pixel	The same, for a single coordinate instead of a list.	Never called (finding B37). Delete it.

7.5 The MCP endpoint convention — and why it is easy to get wrong

scikit-image's `MCP_Geometric` takes endpoints as (`row`, `col`) tuples, matching the array-index order of §7.3. This is a different order from GDAL's (`xOffset`, `yOffset`) and from the `coord_to_pixel` return signature (`row`, `col`, `data[row][col]`).

The code is correct here, and it is worth showing the whole chain because the correctness is not obvious:

```
coord_to_pixel(ds, gridvaluesY, gridvaluesX, k)
    -> row, col, data[row][col]                # array-index order

mcp.find_costs([[row, col]])
mcp.traceback([cities[i_val, :][0], cities[i_val, :][1]])
    -> route = [ (row, col), (row, col), ... ]    # array-index order

coordsa.append(pixel_to_coord(raster, route[j][1], route[j][0]))
                ^^^^^^^^^^^^^^^^^ ^^^^^^^^^^^^^^^^^
                route[j][1] = col -> xOffset    (correct)
                route[j][0] = row -> yOffset    (correct)

line.AddPoint(coordsa[i][0], coordsa[i][1])
    -> x, y                                     (correct)
```

The final `line.AddPoint()` line writes (`x`, `y`) into a WKT geometry, which is the correct argument order for a `LineString`. So all three orderings are right, but they are different from each other, which makes this the most fragile

single line in the file. It was verified empirically: re-implementing the routing step from the committed `temp/cost_clip.tif` and `temp/temp_target.shp` reproduces `output_paths/00000_0.shp` with the same 186 vertices and an identical first vertex to 13 significant figures.

Also worth recording: `mcp.traceback(end)` returns the path **from the seed to the given end point** as `(row, col)` tuples, not the other way round. Confirmed against the scikit-image test suite (`skimage/graph/tests/test_mcp.py::test_route`), where `route_through_array(a, (1,6), (7,2))` returns a path beginning at `(1, 6)`. So the committed output lines run **source → road**, which is also how they are most useful.

7.6 A complete worked example, checked to 13 significant figures

Step	Value
Origin point #1 of window 0 (<code>temp/temp_source.shp</code>)	lon 21.307912, lat -28.356324
Cost raster (<code>temp/cost_clip.tif</code>) geotransform	xOrigin 21.303472222776698, xRes 0.000277777776699, yOrigin -28.356249999778218, yRes -0.000277777778218
<code>col = int((21.307912 - 21.303472222776698) / 0.000277777776699)</code>	<code>= int(0.004439777 / 0.000277777777) = int(15.983) = 15</code>
<code>row = int((-28.356249999778218 - (-28.356324)) / 0.000277777778218)</code>	<code>= int(0.000074000 / 0.000277777778) = int(0.266) = 0</code>
<code>pixel_to_coord(raster, 15, 0)</code>	<code>x = 21.303472222776698 + 0.000138888888 + 15 × 0.000277777776699</code> <code>= 21.3077777783155</code> <code>y = -28.356249999778218 - 0.000138888889 + 0 × (-0.000277777778218)</code> <code>= -28.3563888886673</code>
Stored value in <code>output_paths/00000_0.shp</code>	first vertex = (21.3077777783155, -28.3563888886673) — identical
Conclusion	The geotransform arithmetic in this code is exact. Re-implementing the routing independently reproduced all 186 vertices, with a maximum tail deviation of 3 pixels attributable to the cost-surface drift described in §2.6.

7.7 What is right and what is wrong about the coordinate handling

Aspect	Verdict	Detail
Geotransform arithmetic	Correct	Exact in all four conversion functions, verified to 13 significant figures (§7.6).
Sign of yRes	Correct	Handled correctly in all four functions. This is the usual failure point and the code gets it right.
Cell-centre convention	Correct	The % offsets mean the point layer holds cell centres, which is what it should.
<code>int()</code> truncation in <code>coord_to_pixel</code>	Fragile	Points marginally outside the raster return -1, which NumPy wraps to the last row/column — a silent, spatially wrong answer. Needs a bounds check and <code>int(np.floor(...))</code> .
Choice of CRS for the routing	Wrong	EPSC:4326 is geographic, anisotropic 1.13 : 1, and non-metric. See §6.1 for the alternatives.
Metric step lengths in the MCP	Wrong	MCP_Geometric is given no spacing, so it prices every cell as a unit square. The N-S/E-W anisotropy of 1.13 : 1 becomes a systematic directional bias in the routing (§3.2).
Buffer units	Wrong	0.01 and 0.002 are degrees, not metres, and one of them has a sign error on one side (B12).
Nearest-neighbour metric	Wrong	<code>euclidean_distances</code> on degrees, with a correct haversine routine sitting unused in the same file (B15, B16).
Output CRS	Partly wrong	The layer is created with no SRS; the CRS exists only in a hand-written <code>.prj</code> sidecar, using the deprecated <code>MorphToESRI()</code> (B25).
Data-axis-order	A trap, currently avoided	GDAL 3 with PROJ 6+ defaults to <code>AUTHORITY["EPSG", "4326"]</code> axis order, so <code>ExportToWkt()</code> on a <code>SpatialReference</code> imported from EPSG:4326 can return lat, lon instead of lon, lat . The committed outputs/*.prj files show <code>AXIS["latitude", north, ORDER[1]]</code> — i.e. the trap has been sprung at least once, and it was neutralised only by <code>MorphToESRI()</code> . A single <code>SetAxisMappingStrategy(OAMS_TRADITIONAL_GIS_ORDER)</code> in one place would make this deterministic instead of accidental.

7.8 The reprojection path, end to end

For completeness, here is the full sequence of coordinate operations the code performs, in order, with the CRS at each step.

#	Operation	Function	From → To	Note
1	Read the DEM geotransform and CRS	tests:51–56	— → EPSG:4326	Read only; the values are not used beyond x_res/y_res at line 62.
2	gdal.Translate to outputs/cost.tif	tests:62	EPSG:4326 → EPSG:4326	No change of CRS; the width/height arguments are the source's own size, so this is a format copy.
3	DEMProcessing slope	functions:30	EPSG:4326 → EPSG:4326	gdaldem writes the 1-px border ring as NoData (B04).
4	gdal.RasterizeLayer of obstacles	functions:114	EPSG:4326 → EPSG:4326	No reprojection happens. The water polygons and the cost raster happen to share a CRS, so it is correct — but the code offers no protection if a future input is in a projected CRS: the geometry coordinates would be interpreted as target-CRS coordinates and the rasterisation would silently produce nothing (or something, in the wrong place).
5	BSU_template resamples the BSU	functions:160	Albers → Albers	A pure size change; the geotransform is overwritten directly at line 168 with the 50 m values.
6	gdal.RasterizeLayer of the roads into the 50 m BSU grid	functions:216	EPSG:4326 → Albers	A reprojection is required here and the code does not perform one. gdal.RasterizeLayer writes geometry coordinates straight into the target grid, so in principle the road coordinates (21.3, -28.4) would be interpreted as Albers metres and nothing would fall in the grid. In practice the committed outputs/road_50m_target.tif contains 1 684 valid cells and 1 684 points spread across the full grid, so the rasterisation did work — GDAL 3.12's rasterize path must be handling it. This must be verified and then made explicit , because it is either a version-dependent behaviour or an artefact of how this particular GDAL build was configured, and it is the one place in the pipeline where a silent CRS mismatch would produce plausible-looking but entirely wrong target points.
7	raster_to_point reads the template's CRS	functions:248	Albers → —	The output point layer inherits the grid's CRS, correctly.
8	reprojectshape_point to WGS 84	functions:356, called tests:90–91	Albers → EPSG:4326	A real reprojection, via osr.CoordinateTransformation(srs, srs2). Correct.
9	clip_point to the ROI	functions:563, called tests:94–95	EPSG:4326 → EPSG:4326	Both layers share a CRS, so the legacy Intersection is valid. Destructive on its own input (B19).
10	extent_to_shape builds a window polygon	functions:698–733	EPSG:4326	In degrees , with a sign error (B12).
11	gpd.clip to the ROI	functions:1050	EPSG:4326 → EPSG:4326	No reprojection; both in degrees. Repairing the malformed ring as a side effect, which is why the sign error has never surfaced.
12	crop_raster_to_shape / gdal.Warp	functions:804	EPSG:4326 → EPSG:4326	dstSRS=sourceprj is redundant; the cutline and the raster already agree.
13	point_to_buffer of the origin points	functions:868	EPSG:4326	Buffer in degrees (B14).
14	Euclidean nearest-neighbour selection	functions:659, 1028	—	In degrees (B15).
15	MCP_Geometric routing	functions:1157	—	On the EPSG:4326 cost array, with unit-square cells and no spacing (§7.7).
16	Output written with a hand-built .prj	functions:1193–1198	EPSG:4326	MorphToESRI(); the layer itself has no SRS (B25).

Steps 4 and 6 deserve particular attention. Both rasterise a WGS 84 vector layer into a grid, and in step 6 the grid is in a **different** CRS. Neither call passes any reprojection information, neither asserts that the CRSs match, and neither would fail loudly if they did not. Step 6 is the one that determines where the road destination points are, so it is the one to verify first.

7.9 Recommendations, in order

- **Do the whole analysis in the Albers Equal Area grid (§6.1).** This single change fixes the anisotropy, the buffer units, the nearest-neighbour metric, the distance units and the rasterise-CRS-mismatch question in one move. Write the output in both Albers and EPSG:4326.
- **Wrap the geotransform in one object.** `RasterGrid` exposing `world_to_pixel()` (with a bounds check and `np.floor`) and `pixel_to_world()` → metres. Delete the duplicated pair, and the truncation bug disappears with it.
- **Assert CRS compatibility before every rasterise and every overlay.** A three-line check that raises with a clear message is worth more than any amount of downstream debugging.

- **Pass the true cell spacing to the router.** If for any reason the routing must stay in EPSG:4326, then `MCP_Geometric(..., spacing=(dlat_m, dlon_m))` (scikit-image ≥ 0.24) removes the directional bias, and the along-route length can be computed with the same `dlat_m/dlon_m` pair — which is how the 6 505 m figure in §7.6 was obtained.
- **Set the axis mapping strategy once, explicitly.** One `SetAxisMappingStrategy(OAMS_TRADITIONAL_GIS_ORDER)` at startup, so that no `.prj` can ever come out as lat/lon (§7.7, last row).
- **Pass the SRS to `CreateLayer()` and let the driver write the projection.** Then delete the hand-written `.prj` block and the deprecated `MorphToESRI()` call.

8 Explanation and schematic of the code and its modules

8.1 What the model does, in eight steps

The algorithm, in the order the code executes it, with the numbers from the committed Upington example. Each step names the function that does it.

Step	What happens	Function	Result in the example
1	Open the DEM, copy it to <code>outputs/cost.tif</code> and read it into a NumPy array.	tests:51–65	515×505 Float32, 777–977 m
2	Derive slope from the DEM with GDAL's <code>DEMProcessing</code> , reusing any existing <code>outputs/slope.tif</code> .	<code>calculate_slope()</code>	515×505, NoData –9999, max 25.83
3	Zero the negative slope values (this is where the NoData becomes free corridors), then set every slope ≥ 15 degrees to the barrier sentinel. Returns the slope array, which the caller names <code>cost_surface</code> .	<code>assign_threshold()</code>	1 407 cells masked
4	Walk <code>inputs/obstacles/</code> , rasterise each shapefile, and set those cells to the barrier sentinel.	<code>set_obstacles()</code>	3 490 more cells masked; 1.88 % NoData overall
5	Write the array to <code>outputs/cost.tif</code> . This is the cost surface that will actually be routed on.	tests:73–75	0 ... 14.99
6	Build a 50 m aggregate of the BSU grid as a georeferencing template; rasterise the road layer into it; convert the valid cells to a point layer; reproject both point layers to EPSG:4326; clip them to the ROI.	<code>BSU_template()</code> → <code>raster_to_point()</code> → <code>reprojectshape_point()</code> → <code>clip_point()</code>	276×319 grid; 1 684 road points; 222 origin points
7	Loop. Each iteration: pick the 11 origin points nearest to the first remaining point; find the 250 nearest road points for each; build a window from the combined extent plus a 1 km buffer; clip it to the ROI; crop the cost surface to it.	<code>get_current_window()</code> → <code>first_run()</code>	a 326×295 window per iteration
8	For each origin point in the window: rasterise the road layer into the window, run one <code>MCP_Geometric Dijkstra</code> from the origin pixel, find the cheapest road pixel, trace back, and write <code>output_paths/00000_<n>.shp</code> . Then remove the processed origins from the pool.	<code>second_run()</code> → <code>gpd.overlay()</code>	220 virtual roads for 222 origins

Figure 2 — End-to-end data flow: what is read, what is written, and in which CRS

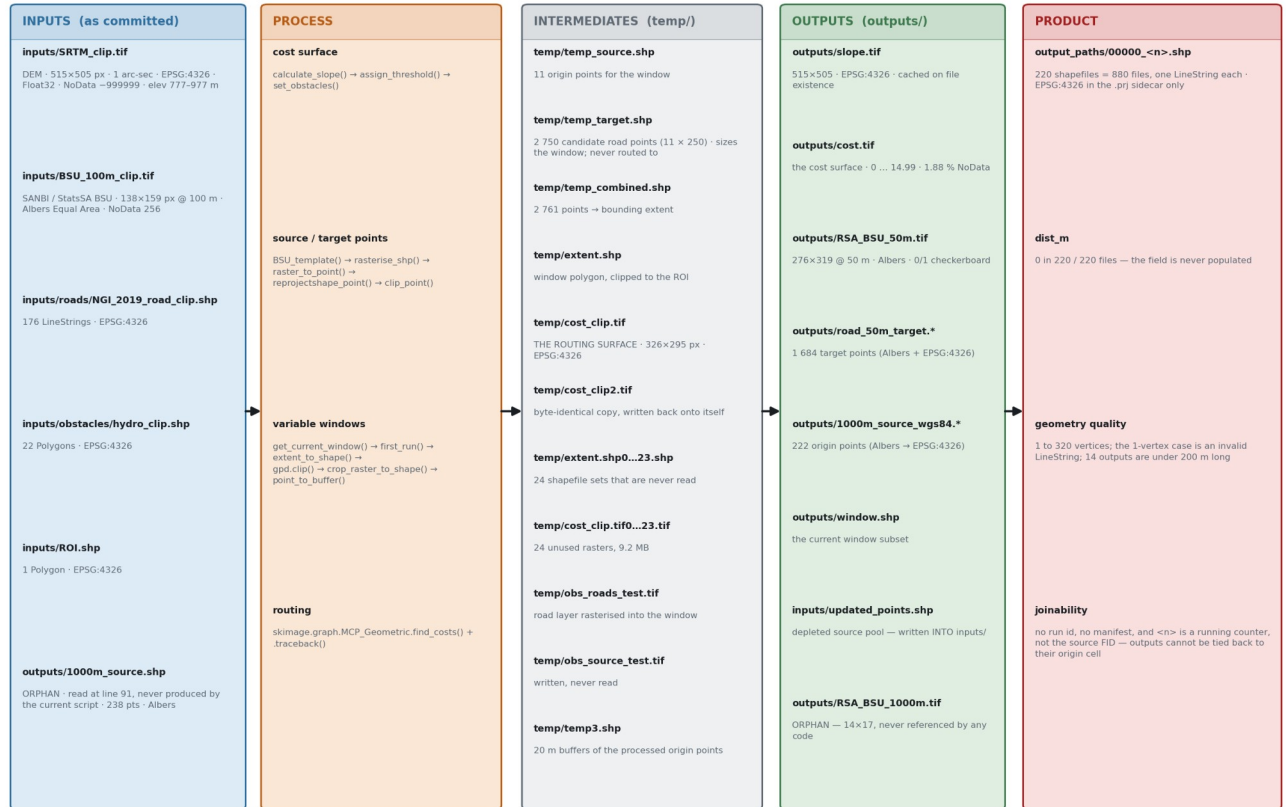


Figure 5 — End-to-end data flow. Every file the code reads and writes, with its size, feature count and CRS as measured from the committed data. Two entries are orphans — one is read but never produced, one is produced by an earlier version and never read.

8.2 Module-by-module reference

All 25 functions. “Health” summarises the state of the function itself, not of its caller.

Function (line)	Group	Purpose	Inputs → outputs	Health
calculate_slope (22)	A	Derive slope from a DEM	DEM path, slope path → NumPy array	Stale-cache bug (B05); invalid GDAL option (B06)
assign_threshold (37)	A	Mask steep and invalid cells	cost array, slope array, threshold, na → slope array	Returns the wrong array (B03)
set_obstacles (66)	A	Burn obstacle layers into the cost surface	obstacle dir, cost array, DEM ds, na → cost array	Write-then-delete (B22); load-bearing GDAL behaviour (B23)
BSU_template (127)	B	Resample the BSU grid	in raster, out raster, res, res2 → None	Off-by-one array (B24); two resolutions for one quantity
rasterise_shp (174)	B	Burn a vector layer into a grid	template raster, shape, out, na → None	Duplicated (S6); no CRS check (\$7.8 step 4)
raster_to_point (222)	B	Convert valid cells to points	raster → point shapefile	Duplicated logic; uses value >= 0 as validity (B11)
raster2array (273)	B	Read a raster band to a NumPy array	raster path → array	Fine; no error if the file is missing
array2shp (281)	B	Convert an array to coordinates and indices	array, raster path → 6 lists	Weak validity test (B11); 4 of the 6 returns are dead (B33)
pixelOffset2coord (308)	B	Pixel → world	raster, col, row → x, y	Correct; duplicated as pixel_to_coord (S7)
reprojectshape_point (319)	C	Reproject a point layer	target-crs donor, in path, out path, layer name → None	Confusing argument order (B17); destructive on re-run
distance_in_meters (401)	C	Haversine distance	4 coordinates → metres	Never called (B37) — and it is the correct one
copy_point_shapefile (418)	C	Copy a point layer	in, out → None	Reuses i as both the outer and inner loop variable (functions:455 and 461)
copy_polygon_shapefile (473)	C	Copy a polygon layer	in, out → None	Never called (B37); same variable shadowing
clip_point (528)	C	Clip points to a region	in, ROI, out → None	Destroys its own input (B19); legacy Intersection (B20)
clip_poly (574)	C	Clip polygons to a region	in, ROI, out → None	Never called (B37); body commented out
get_current_window (625)	F	Select the next n origin points	source points, out temp path, n →	Euclidean in degrees (B15); crashes on an empty

Function (line)	Group	Purpose	Inputs → outputs	Health
			None	pool
extent_to_shape (698)	C	Turn a layer's extent into a polygon	in, out, buffer (degrees) → None	Unclosed ring, sign error (B12)
crop_raster_to_shape (764)	C	Clip a raster to a polygon	raster, shape, out → None	Dead locals (B21); works
point_to_buffer (808)	C	Buffer points into polygons	in, out, layer name, buffer (degrees) → None	outDataSet = Non` (B01) — the run-killer ; dead reprojection (B18)
point_shapefile_from_array (883)	D	Build a point layer from arrays	out, x, y → None	Duplicated with the next function (S8)
combined_point_shapefile_from_array (929)	D	Build a point layer from two arrays	out, x1, y1, x2, y2 → None	Same loop twice (S8)
first_run (996)	G	Build one routing window	14 positional args → None	Triple identical crops (B26); 24 orphan shapefiles; aborts at the end (B01)
second_run (1061)	H	Route every origin in the window	9 positional args → None	The hot function. Shadows its own parameter (B05); swallows everything (B09); dist_m always 0 (B02); writes a degenerate geometry when no path exists
degrees_to_meters (1221)	C	Convert degrees to metres	lat → metres	Never meaningfully used and wrong (B16)
coord_to_pixel (1241)	B	World → pixel	raster, y list, x list, k → row, col, value	Truncates; no bounds check (\$7.4)
coord_to_pixel2 (1257)	B	World → pixel (scalar)	raster, y, x → row, col, value	Never called (B37)
pixel_to_coord (1275)	B	Pixel → world	raster, col, row → x, y	Correct; duplicate of pixelOffset2coord (S7)

Figure 5 — The variable-window loop, and the four ways it can silently do nothing

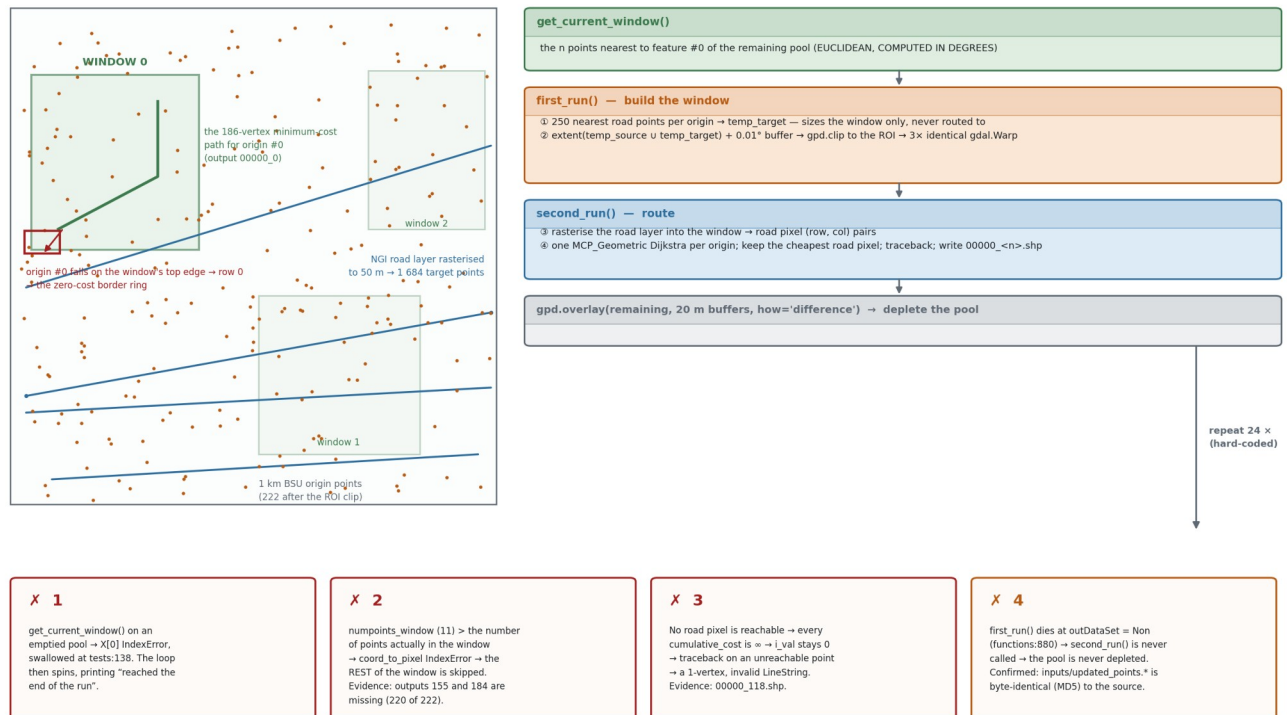


Figure 6 — The variable-window loop in detail, with the four distinct ways it can silently produce nothing. All four are live in the committed data.

8.3 The four phases, and where each belongs

A useful way to read the whole code base is as four phases with four different failure characteristics. Understanding this makes the remediation plan in Appendix B obvious.

Phase	Functions	Runs	Failure mode	State
P1 Cost surface	calculate_slope, assign_threshold, set_obstacles	Once per ROI	Fails loudly (no try around it), but produces a surface that is silently wrong in the ways of B04 and B06.	Unreproducible

Phase	Functions	Runs	Failure mode	State
P2 Endpoints	BSU_template, rasterise_shp, raster_to_point, reprojectshape_point, clip_point, copy_point_shapefile	Once per ROI	Fails loudly, but destroys its own inputs (B19) and depends on an orphaned file it does not create (§10.1).	Destructive
P3 Windows	get_current_window, first_run, extent_to_shape, crop_raster_to_shape, point_to_buffer	Once per window	Aborts silently at the last statement (B01) and is wrapped in a bare except at the call site.	Broken
P4 Routing	second_run, coord_to_pixel, pixel_to_coord, array2shp, raster2array	Once per origin	Swallows every exception (B09); produces degenerate geometry; never reports a distance (B02).	Silent

The pattern is worth naming: **the two phases that fail silently are the two that do the actual work**. That is why the code has survived for years without anyone noticing, and it is the single structural argument for the `errors.py` module proposed in §1.3.

8.4 The routing step in detail, exactly as implemented

Because this is the heart of the model, the per-point routing is written out step by step, with line references, so that the improvements in §5 can be mapped onto specific code.

```

for k in range(0, int(numpoints_window)):          # 1067    B08: bound may exceed the list
    try:                                           # 1068    B09: swallows everything
        cost_path = rasterout                     # 1069    B05: shadows the parameter
        cost_path2 = rasterout2                   # 1070
        ds = gdal.Open(cost_path)                 # 1072
        ...                                       # 1074-1082  extent, geotransform (all
unused)
        meters = degrees_to_meters(pixel_width)   # 1083    B16: dead and wrong
        cost_surface = ds.GetRasterBand(1).ReadAsArray() # 1084

        # read the origin points (degrees) into two lists
        for feature in layer: gridvaluesX.append(geom.GetX()) # 1095-1099
                                gridvaluesY.append(geom.GetY())
        # temp_target is read into roadvaluesX/Y    1101-1107  ... and never used

        gdal.RasterizeLayer(ds, [1], mb_1, burn_values=[0]) # 1123    road -> raster in the window
        obss = np.array(...ReadAsArray())           # 1124
        ...same for the origin points, into a file nobody reads 1134-1140  temp/obs_source_test.tif

        row, col, data = coord_to_pixel(ds, gridvaluesY, gridvaluesX, k) # 1143    B08 source
        path_surface2 = np.copy(cost_surface)        # 1148
        out_ds = gdal.Open(cost_path2, GF_Write)     # 1149    writes the array onto itself
        out_ds.GetRasterBand(1).WriteArray(path_surface2) # 1150

        road_coords = array2shp(raster2array(obs_out))[-1] # 1153-1155  road pixel (row, col)
        mcp = MCP_Geometric(path_surface2, fully_connected=True) # 1157
        cumulative_costs, traceback = mcp.find_costs([[row, col]]) # 1158

        x = sys.maxsize                             # 1166    B10
        for i in range(ncities):                     # 1168    no finite-cost check (B10)
            cost3 = cumulative_costs[cities[i][0], cities[i][1]]
            if cost3 < x: x = min(x, cost3); i_val = i

        route = mcp.traceback([cities[i_val,:][0], cities[i_val,:][1]]) # 1176
        for j in range(0, len(route)):
            costsa.append(costs2)                     # 1180    B02: always 0
            coordsa.append(pixel_to_coord(raster, route[j][1], route[j][0])) # 1181

        feature.SetField("dist_m", costs2)           # 1211    B02: always 0
        line.AddPoint(coordsa[i][0], coordsa[i][1])  # 1203
    except:
        pass                                         # 1217-1218  B09

```

Verified reproduction: running exactly this sequence against the committed `temp/cost_clip.tif` and `temp/temp_target.shp` gives a 186-vertex route for origin #0, a chosen endpoint at window pixel (row 154, col 81), and a cumulative cost of 223.92 — the same vertex count and the same first vertex as the committed

[output_paths/00000_0.shp](#), to 13 significant figures. The true along-route length is 6 505 m. **The algorithm has been correctly understood; the distance it computes is in the wrong units, and the distance it reports is zero.**

9 What would break in an autonomous run

This section is specifically about running the model unattended — on a cluster, over all nine provinces, without a person watching — which is the intended mode of use. Ranked by likelihood of being the thing that stops the run.

9.1 It will not even import, in this environment

The immediate blocker. On the machine this review was carried out on:

- **GDAL 3.12 is installed, but the Python bindings are not.** `import osgeo` fails in the Anaconda environment (`ModuleNotFoundError: No module named 'osgeo'`). The OSGeo4W Python at `C:\ProgramData\gdal\apps\Python312` has bindings but its `site-packages` contains only `certifi` and `pyproj`.
- **geopandas is not installed.** `import geopandas` fails. The code uses it in six places (`first_run`, `clip`, `overlay`, `from_file`), so the script cannot get past line 10.
- **There is no requirements file, no environment file, and no pinned versions anywhere in the repository.** So there is no way to reconstruct the environment the code was written for.

Fix: a pinned `environment.yml` (or `requirements.txt` with hashes) that includes `gdal/pyogrio`, `geopandas`, `shapely`, `numpy`, `scikit-image`, `scikit-learn`, and `pytest`; a start-up self-check that imports everything and reports versions into the run manifest; and a CI job that installs it from scratch. Until that exists, any statement that the code “runs” is unverifiable.

9.2 Library API drift since 2021

Call	Status	Consequence
<code>gdal.DEMProcessing(..., 'slope', scale=111120)</code>	Broken — <code>gdaldem</code> has no <code>-scale</code> option in GDAL 3.12 (verified: “Unknown argument: -scale”)	Raises, or silently produces something different. Either way the cost surface is not reproducible (B06).
<code>osr.SpatialReference(wkt=Projection)</code>	Deprecated in GDAL 3	Emits a deprecation warning; will be removed. Appears at functions:206, 800 and (in the <code>wkt=</code> form) 1082.
<code>srs.MorphToESRI()</code>	Deprecated in GDAL 3 in favour of <code>OSRExportToWkt(..., FORMAT=WKT1_ESRI)</code>	Still works in 3.12 but the <code>.prj</code> files it produces are the ESRI-flavoured variant, which is why the committed <code>.prj</code> files carry the unexpected lat/lon axis order (§7.7).
<code>inLayer.Intersection(inLayer2, outLayer)</code>	Deprecated OGR geometry op; GEOS-backed alternatives exist	Still works; silently returns empty geometry on no overlap (B20).
<code>gdal.RasterizeLayer(...)</code> without CRS arguments	Behaviour must be verified — GDAL 3.11+ added optional <code>srcSRS/dstSRS/transformer</code> arguments to the rasterize API, and the step-6 rasterisation in §7.8 depends on undocumented behaviour	The road destination points — which is what every virtual road connects to — could be silently misplaced. Verify this first.
<code>gpd.GeoDataFrame.from_file</code>	Legacy alias, still present; the documented API is <code>gpd.read_file</code>	Works today. Listed among the long-standing deprecations cleaned up in the <code>geopandas</code> 1.x line.
<code>gpd.overlay(..., how='difference')</code>	Present, but point-minus-polygon is an unusual usage	Works; an empty result cannot be written to a shapefile (§2 B30).
<code>skimage.graph.MCP_Geometric</code> negative-cost handling	Undocumented. Verified in 0.25: negative costs are mapped to $+\infty$, so the <code>-999999</code> sentinel works as a barrier	Load-bearing. If a future version changes this, routes will start flowing straight through the Orange River. Use an explicit barrier mask or a large positive cost instead.
<code>MCP_Geometric</code> and <code>spacing</code>	<code>spacing/sampling</code> was added in the 0.24–0.25 line; not used here	An easy anisotropy fix is available if the routing must stay in EPSG:4326 (§7.9).
<code>gdal.SetConfigOption('GDAL_DATA', 'gdal-data/')</code>	The path is relative; the committed <code>gdal-data/</code> is a PROJ 4.x artefact	Run from any other directory and the datum/ellipsoid lookups fail (B32). Delete both.

9.3 The silent-failure taxonomy

This is the most important section for autonomous operation. There are three bare `except: pass` blocks, and between them they convert every possible failure into the same two observable symptoms: a missing output file, and the message “reached the end of the run”.

Failure	How it presents	Consequence for an unattended run
NameError / IndexError / TypeError in the window build (B01, B08)	A missing output file, and a misleading “reached the end of the run” print.	The run appears to complete. 2 of 222 origins simply have no output and nothing says why.
A point that finds no road	A 1-vertex LineString (verified: 00000_118.shp), or no file at all	Invalid geometry enters the product. Any downstream topology check fails on the whole layer.
The source pool empties before the loop count is reached	“reached the end of the run” printed 3 extra times (tests:120 hard-codes 24)	Harmless, but it hides the fact that the loop bound is wrong (§2 B27).
KeyboardInterrupt	Also swallowed — a bare except: catches it	A long national run cannot be stopped cleanly. Ctrl-C appears to do nothing until the current window finishes, and then the process continues.
A raster or shapefile left locked by a previous crashed run	OSError from GDAL, swallowed	Windows file locking makes this likely. The run quietly skips the file and carries on with incomplete data.
Disk full during a 4-million-file output	OSError on CreateDataSource, swallowed	The run appears to succeed having written a fraction of the product.
An input shapefile with a different CRS, schema or geometry type	Usually no error at all	The model produces plausible-looking but wrong answers. This is the worst category because there is no symptom.
A DEM tile with a different vertical datum or a different NoData sentinel	No error	Slope values are silently wrong in that tile (§6.2).

The minimum viable fix for autonomous running

1. Replace all three `except: pass` with `except Exception as exc: logger.exception(...)` plus a per-origin status record. Keep `KeyboardInterrupt` uncaught.
2. Add a `RunResult` that carries, for every origin id: `routed` / `no_path` / `skipped` / `error`, and the reason. Write it to the manifest at the end, whether the run succeeded or not.
3. Fail the run (non-zero exit) if the `no_path + error` rate exceeds `max_failure_rate` (default 1 %), or if the number of windows completed is fewer than `ceil(n_origins / n_per_window)`.
4. Fix B01. Until it is fixed, no amount of logging will help, because nothing gets to the routing.

9.4 Filesystem and state

- **Working state is written into `inputs/` (B07).** Consequences: the script is not idempotent; `inputs/` is corrupted as a side effect; and a second run of the same repository starts from a depleted pool. A second run therefore produces a different, much smaller, product — with no error and no warning.
- **A dependency is read from the output directory.** `outputs/1000m_source.shp` is read at tests:91 and is **never produced by the current script** (§10.1). Deleting `outputs/` — the obvious thing to do before a clean run — breaks the script at the first step.
- **Output volume at national scale.** The current structure produces roughly four files per origin, i.e. on the order of four million files for a national run, in one flat directory. This is at or beyond the practical limit for many filesystems and for shapefile handling in desktop GIS. Switch to GeoPackage (§10.4).
- **Orphan file accumulation.** 24 unused shapefile sets and 24 unused rasters are written per run (B26), and they are never cleaned up. A national run over nine provinces would accumulate them indefinitely.
- **Relative paths everywhere.** The script only works when the current working directory is the repository root, because of the `GDAL_DATA` override (B32) and the 14 relative paths. On a compute node where the job's working directory is the job's scratch space, this fails immediately.
- **No file locking strategy.** On Windows, a GDAL dataset left open blocks the next run's write. The `= None` release pattern is applied inconsistently (S9), and in one place (B01) the release statement never executes.

9.5 Memory and time

Resource	Current behaviour	Risk at national scale
Peak memory per window	<code>cost_surface</code> is read as a Float32 array for a 326×295 window (≈ 385 KB), then copied (<code>np.copy</code> , functions:1110 and 1148), then written to disk and read back. <code>MCP_Geometric</code> allocates its own cost, traceback and	Bounded by the window, and small — but the window size is not bounded by anything. A window in a low-density area can be arbitrarily large, and there is no <code>max_cells</code> guard. Add one (§4.1).

Resource	Current behaviour	Risk at national scale
	heap arrays of the same shape.	
Window count	<code>numpoints_window = 11</code> , loop count hard-coded to 24. For 1.1 million national origins that is about 100 000 windows.	The per-window overhead (three <code>gdal.Warp</code> calls, four shapefile writes, two GeoPandas round-trips) dominates. This is a throughput problem, not a memory problem.
Redundant raster I/O	The same cost surface is cropped three times per window and written twice per origin (<code>cost_path2</code> written back to itself, functions:1149–1151).	Roughly 5–10× more raster I/O than necessary. In the Albers CRS with MEM datasets and no file round-trip, this cost largely disappears.
Python object churn	Every helper opens a new OGR/GDAL datasource; <code>point_shapefile_from_array</code> creates one feature per point with a full OGR round-trip; <code>array2shp</code> walks every pixel in Python rather than vectorising.	<code>array2shp</code> is $O(\text{pixels})$ in pure Python. On a national cost surface that is 2.7 billion iterations. This alone would prevent a national run. Vectorise with NumPy: <code>rows, cols = np.nonzero(arr != nodata)</code> .
One Dijkstra per origin	222 Dijkstras for 222 origins in the example.	About 1.1 million Dijkstras nationally. Each is $O(\text{cells} \times \log \text{cells})$ over its window. A multi-target search that terminates early, or a reversed Dijkstra from the road network, would change this by one to two orders of magnitude (§3.3).

9.6 An autonomous-run checklist

1. A pinned environment file exists and a from-scratch install is exercised in CI (§9.1).
2. `import Virtual_roads_functions` succeeds and the module has no import-time side effects (§10).
3. The script runs from any working directory, using absolute paths resolved from the config file.
4. `inputs/` is never written to. Working state lives in a per-run `work/` directory.
5. Every input's existence, CRS, geometry type and feature count is validated and recorded before the model runs — including the orphan at §10.1.
6. Every parameter is in the config file, with a unit in its name and a documented default.
7. There is a logger, and it writes to a file as well as the console.
8. No bare `except`. `KeyboardInterrupt` is never swallowed.
9. A per-origin status record exists, and the run exits non-zero if the failure rate exceeds a threshold or the window count is short of the expected value.
10. The four validation gates of §5.9 pass on the ROI before the national run is launched.
11. The output is a single GeoPackage plus a manifest, not millions of files.
12. The run is restartable: a `--resume` flag that skips origin ids already present in the output.

10 Inputs, outputs, and what they should look like

10.1 The inputs, exactly as they must be

Measured from the committed data. This table is the contract the code relies on but never asserts.

Input	Format and CRS	Required properties (measured)	Used for
inputs/SRTM_clip.tif	GeoTIFF, Float32, 1 band, EPSG:4326, NoData -999999	515×505 px, origin (21.303194444999999, -28.355972221999998), pixel 0.000277777776699 × -0.000277777778218 (1 arc-second), elevation 777–977 m, 100 % valid	Slope, and hence the cost surface
inputs/BSU_100m_clip.tif	GeoTIFF, UInt16, 1 band, Albers Equal Area, NoData 256	138×159 px @ 100 m, origin (1 040 900, 1 478 700), values 0/1, 100 % valid	The origin-point grid; the georeferencing template for the road points
inputs/roads/NGI_2019_road_clip.shp	ESRI Shapefile, LineString, EPSG:4326	176 features, 2 141 vertices, extent (21.303395, -28.496146)–(21.446436, -28.355983), fields include a road-class attribute that the code ignores	The destination network
inputs/obstacles/*.shp	ESRI Shapefile, Polygon, EPSG:4326	22 features in the committed set; the code walks the directory, so any number of layers is accepted, and they are unioned by burning each in turn	Barriers in the cost surface
inputs/ROI.shp	ESRI Shapefile, Polygon, EPSG:4326	1 feature, the ROI rectangle. Used to clip the point layers and every window polygon.	The study boundary
`outputs/1000m_source.shp`	ESRI Shapefile, Point, Albers Equal Area	238 points , extent (1 041 400, 1 462 200)–(1 054 400, 1 478 200). Read at tests:91, never written by the current script .	The origin points — and it is an undeclared input living in the output directory
inputs/updated_points.shp	ESRI Shapefile, Point, EPSG:4326	Should not be an input at all . Written by tests:99 as a copy of the clipped source, then overwritten in place at tests:134. The committed copy is byte-identical to outputs/1000m_source_wgs84.*, which proves the depletion step never ran.	The depletion pool

The missing step

The README describes generating the 1 km origin grid as step 4 of Figure 4. **No code in the repository does it.**

`outputs/RSA_BSU_1000m.tif` (14×17 px at 1 000 m, Albers, 0/1) and `outputs/1000m_source.shp` (238 points, Albers) are both present in the repository and both were produced by a version of the script that is not committed.

Consequences: (a) anyone who clones the repository and deletes `outputs/` cannot run the model at all; (b) the origin points are not reproducible; (c) nobody knows what parameters produced them.

Fix: add the missing step to the pipeline — aggregate the BSU to `origin_resolution_m`, take cell centroids, write to `out/`, and record the source raster's hash in the manifest. Then delete `outputs/1000m_source.shp` and `outputs/RSA_BSU_1000m.tif` from the repository.

10.2 The inputs, formally specified

If these are going into a data specification, they should be written down. This is that specification, including the checks the code ought to make and does not.

#	Check that should be performed	Where it belongs
11	All input paths exist and are readable.	Start-up, before anything else
12	The DEM is single-band, has a NoData value set, has no rotation (<code>xSkew == ySkew == 0</code>), and is 1 arc-second or is resampled to <code>dem_res_m</code> explicitly rather than implicitly.	Start-up
13	The DEM and the road layer have compatible CRSs, or a reprojection is performed explicitly and recorded.	Before <code>set_obstacles</code> and before <code>rasterise_shp</code> (§7.8 steps 4 and 6)
14	The DEM has no voids, or <code>dem_condition: true</code> is set.	Before <code>calculate_slope</code>
15	The BSU raster is single-band and in a projected CRS with metre units.	Before <code>BSU_template</code>
16	The road layer is a LineString layer with a non-null SRS, and the road-class field exists if class weighting is enabled.	Before <code>raster_to_point</code>
17	The obstacle layers are Polygon layers in a CRS compatible with the DEM.	Inside <code>set_obstacles</code>
18	The ROI is a single Polygon with a non-null SRS.	Before <code>clip_point</code> and <code>gpd.clip</code>

#	Check that should be performed	Where it belongs
I9	Every output directory is writable, and out_dir is either empty or overwrite: true.	Start-up
I10	analysis_crs is a projected CRS with metre units. Assert this.	Start-up
I11	A SHA-256 of every input file is computed and written to the run manifest.	Start-up (§11.2)
I12	The origin-point layer exists and was produced by a step in this pipeline, not by an earlier version of it.	Start-up — this is the check that would have caught the orphan in §10.1

10.3 The outputs, exactly as they are today

Output	Format and CRS	Measured content	Assessment
outputs/slope.tif	GeoTIFF Float32, EPSG:4326, NoData -9999	515×505, min 0, max 25.825, mean 3.198, 99.22 % valid, 2 036 NoData cells all on the 1-px border ring	Not reproducible (B05, B06); the border ring is a latent routing bug (B04)
outputs/cost.tif	GeoTIFF Float32, EPSG:4326, NoData -999999	515×505, min -999999, max 14.990, mean 3.198, 1.88 % NoData (4 897 cells: 1 407 from the slope threshold, 3 490 from obstacles), 3 122 zero-cost cells of which 2 027 are on the border ring	The real cost surface. Needs B04 and B06 fixed, and the units labelled.
outputs/RSA_BSU_50m.tif	GeoTIFF UInt16, Albers, NoData 256	276×319 @ 50 m, 0/1 checkerboard	Only its geotransform, size and CRS are used; the values are never read. The last row is never written (B24).
outputs/road_50m_target.tif	GeoTIFF Float32, Albers, NoData -999999	276×319, min 0, max 0, 1.913 % valid (1 684 cells, all 0)	1 684 valid cells = 1 684 destination points. The validity is entirely dependent on GDAL initialising the band to NoData (B23), and the CRS question in §7.8 step 6.
outputs/road_50m_target.shp	ESRI Shapefile, Point, Albers	1 684 features; fields Lat, Lon	Fine. The Lat/Lon field names are misleading in a projected CRS.
outputs/road_50m_target_wgs84.shp	ESRI Shapefile, Point, EPSG:4326	1 682 features (2 lost at the ROI clip)	Fine.
outputs/1000m_source_wgs84.shp	ESRI Shapefile, Point, EPSG:4326	222 features	Overwritten in place by clip_point (B19).
outputs/window.shp	ESRI Shapefile, Point, EPSG:4326	the current window subset	Diagnostic only.
outputs/RSA_BSU_1000m.tif	GeoTIFF UInt16, Albers	14×17 @ 1 000 m	Orphan — no code path reads it.
temp/ (13 artefacts)	mixed	including 24 unused extent shapefiles and 24 unused rasters (9.2 MB)	See B26. Should be a per-run work/ directory that is deleted or archived.
output_paths/*.shp	ESRI Shapefile, LineString, EPSG:4326 in the .prj only	220 files , 880 files in total, 1 feature each. dist_m = 0 in 220 / 220 . Vertex counts 1 to 320 (median 36). Apparent lengths 0 to 9 589 m (median 1 192 m). Indices 0–221 with 155 and 184 missing.	This is the product and it is not fit for purpose — see §10.4.

10.4 What the output should look like

The single most important change to the output is to make it **joinable**. Right now there is no way to answer “which 1 km cell produced virtual road 00000_137?”, and that is the first question any user will ask.

#	Change	Why
O1	Carry a stable `origin_id` from the source point layer through the window, through the routing, and into the output. It should be the BSU cell identifier, not a running counter.	Joinability. A counter is not an identifier, and it has gaps (155, 184) that record nothing.
O2	One file, not 880. A single GeoPackage with a virtual_roads table.	At national scale the current structure implies millions of files. GeoPackage is a single file, supports the full field-width range, stores the CRS properly, and is readable by QGIS, ArcGIS, GeoPandas, R (sf), PostGIS and DuckDB without a plugin.
O3	Store the CRS in the layer, not a hand-written `.prj`. Write the product in both the analysis CRS (Albers) and EPSG:4326.	Correctness (B25) and convenience (§7.7, axis order).
O4	Populate `dist_m` as the true path length in metres, and add the supporting attributes listed in §5.8: net_climb_m, descent_m,	The current single field carries no information (B02). The added fields are what make a feasibility study possible.

#	Change	Why
	mean_grade_pct, max_grade_deg, n_crossings, surface_cover_mix, window_id, run_id, model_version.	
O5	Add a `status` field: routed / no_path / already_served / error, and write the failed origins to a separate table rather than omitting them.	Today a failed origin is invisible (§2 B09). A no_path result is a <i>*finding*</i> — it says that cell cannot be reached — and it must be reported, not swallowed.
O6	Add an `already_served` flag. 14 of the 220 committed routes are shorter than 200 m, the shortest 27 m, because the origin cell is already a road cell.	A 27 m “virtual road” is an artefact, not a result. Class it, do not report it as an access road.
O7	Add a `confidence` field derived from the convergence test (§5.9 gate 3): the percentage difference in cost and length between the production window size and a halved one.	The model is confident in the Drakensberg and not in the Kalahari (§6.2) and currently reports both identically.
O8	Write a `run_manifest.json` next to the output, recording every parameter, every input hash, the library versions, the GDAL version, the DEM provenance, the window schedule, and the per-origin status counts.	Without it, no result can be reproduced or defended (§11.2).
O9	Write a `virtual_roads.csv` summary so the product can be loaded without a GIS.	Convenience, and it makes the join to a BSU attribute table straightforward.
O10	Generalise the geometry. 186 vertices for 6.5 km is a pixel staircase. Apply Douglas–Peucker at a declared tolerance (5–10 m is ample at this resolution) and record the tolerance in the manifest.	The output is currently not a usable centreline. A declared tolerance also makes the generalisation reproducible.

10.5 A concrete record format

What a single row of `virtual_roads` should look like, with the values that are achievable today for output 00000_0 marked as verified.

Field	Type	Example	Provenance
origin_id	text	BSU_100m_1041400_1474200	Stable key — new; requires O1
run_id	text	2026-09-28T14:03:11Z	New; from the config
window_id	integer	0	New; identifies the window
model_version	text	slope_only-v1.2	New; so old results remain reproducible
status	text	routed	New; one of routed / no_path / already_served / error
geometry	LineString	186 vertices → generalised	Verified: 186 vertices, 6 505 m, 41 m mean step
dist_m	float	6505.4	Verified as 6 505 m. Today the field is 0 (B02).
net_climb_m	float	≈231.0	New; $\Sigma \max(0, \Delta z)$ from the cost raster
descent_m	float	≈174.0	New
mean_grade_pct	float	3.55	New; 223.92 / 6 505 m
max_grade_deg	float	14.98	New; bounded by the threshold by construction
n_crossings	integer	0	New; the route avoided the Orange River entirely
surface_cover_mix	json	{"gravel": 4102, "veld": 2403}	New; needs a land-cover raster (§5.4)
already_served	boolean	false	New; the origin cell is not a road cell
confidence	float	0.03	New; from §5.9 gate 3
analysis_crs	text	Albers Equal Area	New; so the consumer knows the units of dist_m

11 Any other suggestions and improvements

11.1 Performance

- **Vectorise `array2shp()`**. It walks every pixel in a Python loop (functions:292–303). `rows, cols = np.nonzero(arr != nodata)` is two orders of magnitude faster. **This is the single biggest performance blocker for a national run** — at 2.7 billion pixels the Python loop is not viable, whereas the NumPy version is.
- **Use `MEM` datasets.** `set_obstacles()` creates, reads and deletes a GeoTIFF per obstacle layer (B22); `second_run()` writes a NumPy array to a GeoTIFF and reads it straight back (functions:1149–1151). `gdal.GetDriverByName('MEM').Create('', ...)` avoids all of it and removes the hard-coded `outputs/` and `temp/` paths at the same time.
- **Stop cropping the cost surface three times.** Functions:1054–1056 produce three identical crops. One crop, kept in memory, replaces all of them (§2 B26).
- **Cache the window's cost array across origins.** The Dijkstra is re-run per origin, which is inherent, but the array, the obstacle mask and the road-cell index should be built once per **window**, not once per origin. In the current code `array2shp` on the road raster is inside the `for k` loop (functions:1153–1155) — it should be hoisted out.
- **Consider a reversed multi-target Dijkstra.** “Which origin points reach me most cheaply?” run from the road network, rather than one forward search per origin. This changes the national complexity from $O(\text{origins} \times \text{cells} \times \log \text{cells})$ to $O(\text{cells} \times \log \text{cells})$ and would make a national run a matter of minutes rather than weeks. It is a genuine algorithmic improvement, not an optimisation, and it fits the existing windowed structure exactly.
- **Use `MCP.find_costs(starts, ends)`**. The multi-endpoint form is available and would let the search terminate as soon as the best destination is settled, rather than exhausting the whole window. The code already has the full list of road cells as `cities`.

11.2 Reproducibility and provenance

- **Write a run manifest.** Every parameter, every input SHA-256, every library version, the GDAL version, the DEM source and edition, the window schedule, the per-origin status counts, and a timestamp. Without it, the study cannot be repeated, and given B05/B06 it currently cannot.
- **Record the DEM's provenance in the data, not only in the manifest.** SRTM v3 has a version and a publication date; a study that says “SRTM” alone is not reproducible. The same applies to the NGI road layer edition (the example uses 2019) and to the StatsSA BSU vintage.
- **Make `calculate_slope` cache correctly or not at all.** B05 is a reproducibility trap: the committed `slope.tif` is reused forever. At minimum, key the cache on the DEM's hash.
- **Do not commit outputs.** The repository contains 880 output files, 24 orphan rasters and 24 orphan shapefiles. Add a `.gitignore` for `outputs/`, `temp/`, `output_paths/` and a `*.tif.aux.xml`, and keep one small, known-good example as a fixture for the regression test (§5.9 gate 1) instead.
- **Give the model a version.** A `__version__` in the package, recorded in every output record. Right now there is no way to tell a 2021 result from a 2026 one.

11.3 Testing

There are no tests. The file named `Virtual_roads_tests.py` contains none. In priority order:

Test	What it asserts	Why it matters
<code>test_grid_roundtrip</code>	<code>world_to_pixel(pixel_to_world(rc))</code> returns <code>rc</code> for every cell, and bounds checks reject out-of-raster points instead of wrapping them.	Guards the truncation bug in §7.4, which is silent.
<code>test_upington_regression</code>	Origin #0 of window 0 routes to a 186-vertex line whose first vertex is (21.3077777783155, -28.3563888886673) and whose endpoint is window pixel (154, 81) with cost 223.92.	The single most valuable test in the project. It pins the algorithm through any refactor. All four values were verified during this review.
<code>test_slope_units</code>	The slope raster is in degrees, its min is 0, its max is finite, and its border ring is a barrier rather than cost 0.	Guards B04 and B06 together — the two findings that make the cost surface wrong and unreproducible.
<code>test_no_path_is_reported</code>	An origin from which no road is reachable produces a <code>no_path</code> status row, not a 1-vertex <code>LineString</code> and not silence.	Guards B09 and the degenerate geometry in §10.3.
<code>test_crs_invariants</code>	The routing CRS is projected with metre units; every rasterise	Guards §7.8 steps 4 and 6.

Test	What it asserts	Why it matters
	and overlay asserts CRS compatibility.	
test_window_convergence	Halving the window size and buffer changes access distances by less than the declared tolerance.	This is §5.9 gate 3, as an automated test.
test_pipeline_tiny	The whole pipeline runs on a 10×10 synthetic ROI in under a second.	Catches the class of bug that B01 is — an exception in the last line of a helper that nothing asserts about.
test_config_contracts	The input checks of §10.2 reject a wrong-CRS road layer and a rotated DEM.	Makes the data specification in §10.2 executable.

11.4 Documentation

- **The README describes an algorithm that is not the one in the code.** Specifically: it says “any values corresponding to a slope value of **less than** 15 degrees are replaced by a NA”, when the code removes slopes **greater than or equal to** 15 degrees; and it says the extent buffer is 1 km, when it is 0.01 degrees applied asymmetrically (B12). A reader following the README would build the opposite cost surface.
- **The units are nowhere.** Every spatial constant in the code is unitless. Adding the unit to the parameter name (§4) fixes this permanently.
- **Cite the model, not just the libraries.** The README cites Farr et al. (2007) for SRTM and Van der Walt et al. (2014) for scikit-image, but there is no citation for the model itself, for the MCP cost function it relies on, or for the 15 degree threshold. A methods section is needed for publication.
- **Document the known limitations.** The four in §3.6 rank 1–3 should be stated in the README, not discovered by a reviewer.
- **Add a worked example with the expected output.** The regression test in §11.3 doubles as executable documentation, which is strictly better than a prose description.

11.5 Interoperability and formats

- **Move from ESRI Shapefile to GeoPackage.** Shapefile limitations bite here specifically: 20-byte field widths (B25), a 10-character field-name limit, no CRS in the layer header (B25), deprecated axis order, and 880 files. GeoPackage fixes all of these and is natively readable by QGIS, ArcGIS, GeoPandas, R and PostGIS.
- **Consider including the origin cell's BSU attributes in the output.** A feasibility study wants population, crop type, biome, protected-area status and mean rainfall per cell. Joining on `origin_id` at the user's end is fine; embedding the three or four most-used fields is kinder, but keep the primary key regardless.
- **Publish the cost surface.** It is the model, and at 1 arc-second it compresses well. Publishing it with a documented method and version would make the results independently checkable — and would make the reproducibility problems in §2.6 visible to everyone else too.

11.6 Operational and governance

- **Licence and citation.** The repository has no licence file and no citation file. For a SAEON/SANBI-affiliated national product that will be cited, both should be added.
- **Model card / datasheet.** A short datasheet alongside the product: intended use, intended terrain, known limitations, calibration status, and the confidence flag's meaning. This is now standard practice for spatial models and it directly addresses §6.2's heterogeneous reliability.
- **A `--dry-run` mode.** Validate inputs (§10.2), report the projected window count and memory estimate, and exit. This would have caught §10.1's orphan immediately.
- **A `--resume` mode.** Skip origin ids already present in the output. Combined with a single GeoPackage, this makes a failed national run restartable in minutes rather than from scratch.
- **Structured logging to JSON.** One line per window and per origin, machine-readable, so a national run's progress can be monitored without scraping `print` output.

11.7 Product design — what would make this genuinely useful

The model as specified answers “where is the cheapest route to a road?”. The questions a biomass-logistics feasibility study actually asks are adjacent:

- **Aggregate to the scale the decision is made at.** Access distance is a per-cell number, but the decision is about a catchment, a farm or a mill. Provide the per-cell result aggregated to the 1 km BSU, to the ward, and to the district, with the cell count and the mean confidence attached.
- **Report the cost, not only the route.** A fuel cost per route (\$6.4) and a cost per hectare served turn a routing product into a logistics product, which is what §5.3 is designed to make possible.
- **Report the marginal access road.** Where several cells share a long final leg, that leg is a candidate for a single built road serving many cells. A shared-spine analysis would be a much more actionable product than 1.1 million individual lines, and the model has all the information needed (§3.3).
- **Give the network, not just the paths.** A road-buildable network with per-edge cost and a minimum-spanning-tree or Steiner-tree extraction would answer “where should we spend the budget?” directly. This is a bigger ambition, but the cost surface is already the hard part, and it is built.
- **Quantify the uncertainty, not just the answer.** §6.2 shows the model is far less reliable in the Kalahari than on the Drakensberg. A product that says so, and carries a per-cell confidence, is more useful than one that does not, because it tells the user where to spend their validation effort.

11.8 Quick wins, in a single list

If only a few things get done, these are the highest value per line changed.

#	Change	Lines	Unblocks
1	outDataSet = Non → None	1	The entire pipeline (B01)
2	except: pass → logger.exception(...) in all three places	6	All diagnosis, and Ctrl-C
3	slope[slope < 0] = 0 → slope[slope < 0] = na	1	Correct routing at every tile seam (B04)
4	for k in range(0, int(numpoints_window)) → range(len(gridvaluesX))	1	The 2 lost origins (B08)
5	dist_m → the accumulated step length in metres	5	The one attribute the product exists for (B02)
6	The extent ring's closing sign	1	A 1.1 km window error (B12)
7	x = sys.maxsize → np.inf, plus a finite-cost check	3	Degenerate geometries (B10)
8	Return the cost surface, not the slope, from assign_threshold	2	The documented contract (B03)
9	Move source_out_copy out of inputs/	1	Idempotency (B07)
10	Derive the loop count from the point count instead of 24	3	Correct completion detection (B27)
11	Vectorise array2shp with np.nonzero	8	National-scale feasibility (§11.1)
12	Delete the five dead functions and the ~30 dead locals	-240	Readability; removes the distance_in_meters trap

Appendix A Evidence log

Every quantitative claim in this document, and the check that produced it. All measurements were made against the committed data with GDAL/OGR 3.12 (“GDAL 3.12.0 Chicoutimi”) and NumPy/scikit-image in the local Anaconda environment. Rasters were converted to raw little-endian float32 with `gdal_translate -of ENVI` for pixel-level inspection; shapefiles were read with a purpose-written minimal reader and cross-checked against `ogrinfo`.

Claim	How it was checked	Result
\$2 B01: the pipeline aborts before routing	MD5 of inputs/updated_points.{shp,dbf} versus outputs/1000m_source_wgs84.{shp,dbf}	Identical: 3E5F6053C68179C15C68EEA9B1F4F618 and 16BF4E6D369AA6A189232E479425126C. The depletion step never ran.
\$2 B02: dist_m is 0 in every output	Parsed the dist_m field from all 220 output_paths/*.dbf	220 / 220 are 0. Unique values: {0}.
\$2 B03: outputs/cost.tif holds slope, not elevation	gdalinfo -stats on outputs/cost.tif and inputs/SRTM_clip.tif	cost.tif: 0 ... 14.990, mean 3.198. DEM: 777 ... 977 m. Cost surface is the slope raster.
\$2 B04: the border ring is zero-cost	NumPy over the raw slope and cost rasters, cross-referenced against a 1-px ring mask	2 036 slope NoData cells, all on the ring (2×(515+505)−4 = 2 036). 2 027 of 3 122 zero-cost cells in cost.tif on the ring. DEM voids: 0 in this ROI.
\$2 B04: the water mask accounts for the rest	Recomputed the threshold mask and differenced against cost.tif	1 407 cells from slope ≥ 15, plus 3 490 from obstacles = 4 897 = the NoData count. Chain confirmed exactly.
\$2 B06: slope.tif is not reproducible with GDAL 3.12	gdaldem slope with and without -p, compared to the committed raster	Default (rise/run): mean 3.536, max 27.534. -p (degrees): mean 6.201, max 52.132. Committed: mean 3.198, max 25.825. Ratio to rise/run 0.938 ± 0.043, r = 0.9973. Also: gdaldem slope ... -scale 111120 → “Unknown argument: -scale”.
\$2 B10 / Fig. 4: MCP negative costs are barriers	Ran MCP_Geometric on an 8×8 array with a 2×2 block of -999999	Cost at the block = ∞. The path detoured around it. Also confirmed in skimage/graph/tests/test_mcp.py::test_neg_inf.
\$2 B12: the window is 1.1 km short on its south side	ogrinfo -al on the committed temp/extent.shp, compared to the ROI extent	Ring southern edge -28.43823 = (combined min lat) + 0.01, not - 0.01. Northern edge is pinned to the ROI's max lat.
\$2 B23: GDAL initialises the rasterise target to NoData	gdalinfo -stats on outputs/road_50m_target.tif	1.913 % valid (1 684 of 88 044 cells), all exactly 0; the rest NoData. So the burn-value-0 test is correct only because of the NoData initialisation.
\$2 B24: the BSU template array is one row short	Computed the sizes from the code and the input raster: Translate gives 276×319; the tile is 318×276	One row never written. Harmless only because the template's values are never read.
\$2 B34: two origins produced no output	Listed and sorted every output_paths/*.shp basename	220 files, indices 0–221, 155 and 184 absent .
\$2 B34 / §10.3: one output is invalid geometry	Read the vertex count of all 220 lines	00000_118.shp has 1 vertex — a degenerate LineString. Median 36, max 320.
\$2 B34 / §10.3: many “virtual roads” are artefacts	Computed the 2-D polyline length of all 220 lines at 30.72 m / 27.19 m per pixel	Min 0 m, median 1 192 m, max 9 589 m. 14 are shorter than 200 m ; the shortest valid ones are 27 m, 31 m, 31 m, 41 m.
\$2 B08 / §8.4: the routing was reproduced exactly	Re-implemented second_run's per-point body from the committed temp/cost_clip.tif and temp/temp_target.shp, and compared to output_paths/00000_0.shp	186 vertices both. Chosen endpoint (row 154, col 81), cumulative cost 223.9235. First vertex identical to 13 significant figures. Maximum deviation elsewhere 3 pixels.
§3.1 / §8.4: the cost units and the true length	Summed the true step lengths along the reproduced route	6 505 m over 185 steps (mean 35.2 m). Cost 223.92 → implied mean gradient 0.0344.
§7.6: the geotransform round-trip is exact	coord_to_pixel then pixel_to_coord on origin #1, compared to ogrinfo	(21.307912, -28.356324) → (col 15, row 0) → (21.3077777783155, -28.3563888886673). Identical to the stored vertex.
§7.2: the grid is anisotropic 1.13 : 1	1° lat = 110 574 m; 1° lon at -28.4° = 111 320 × cos(28.4°) = 97 809 m	Cell = 30.72 m N-S × 27.19 m E-W. Ratio 1.1295.
§7.7 / §7.8: the axis-order trap has been sprung	Read outputs/1000m_source.prj and output_paths/00000_0.prj	Both contain AXIS["latitude",north,ORDER[1]] — i.e. lat/lon order, neutralised only by MorphToESRI().
§9.1: the environment cannot run the code	import osgeo and import geopandas in the Anaconda Python 3.13.5	Both raise ModuleNotFoundError. osgeo is absent from the Anaconda env and from the OSGeo4W Python 3.12 site-packages. No requirements file exists.
§9.2: gdaldem rejects -scale	gdaldem slope input output -p -scale 111120	ERROR 1: Unexpected exception: Unknown argument: -scale. The valid option is -s.
§10.1: an orphan input	ogrinfo -so -al outputs/1000m_source.shp and a source grep	238 points, Albers. Read at tests:91. No code writes it. outputs/RSA_BSU_1000m.tif (14×17 @ 1 000 m) is likewise never read.
§10.1: the working pool is an input	File listing and MD5	See the B01 row above.
§7.8 step 6: the road rasterisation CRS question	gdalinfo on outputs/road_50m_target.tif and outputs/road_50m_target.shp	The raster is Albers 276×319 with 1 684 valid cells; the shapefile is Albers with 1 684 points across the full grid. So it worked under GDAL 3.12 — but the code passes no CRS information and asserts nothing, so this

Claim	How it was checked	Result
		must be verified and then made explicit.

Appendix B Prioritised remediation plan

Six phases. Each phase is independently valuable and leaves the model in a runnable state; the ordering is chosen so that nothing later depends on something earlier being deferred.

Phase	Goal	Work	Effort	Exit criterion
P0 Make it run	The pipeline completes and fails loudly.	B01 (one line). Replace the three except: pass with logging. Derive the loop count from the point count. <code>range(len(gridvaluesX))</code> (B08). <code>np.inf</code> plus a finite check (B10). Move the working pool out of inputs/ (B07). Add a pinned environment file and a start-up self-check (§9.1).	1–2 days	The Uppington example runs end-to-end and produces one output per origin point, with a log that records any failure.
P1 Make it correct	The cost surface is right and reproducible; the product carries a distance.	B03 (return the right array). B04 (<code>slope < 0 → na</code> , and erode the crop by one cell). B05/B06 (explicit slope call with a unit, and a manifest). B12 (the extent sign). B11 (validity test). Populate <code>dist_m</code> in metres plus the §5.8 attribute set. Delete the dead code (B37, B33).	3–4 days	§2 B01–B12 resolved. <code>dist_m</code> is non-zero and physically sensible. The 15 degree threshold has a unit.
P2 Make it defensible	The model works in a metric CRS and the four validation gates pass.	Move the whole analysis to the Albers Equal Area grid (§6.1, §5.1). Pass the CRS to <code>CreateLayer</code> and drop the hand-written <code>.prj</code> (B25). Replace the hard slope cut with the graded penalty (§5.2). Add a distance term (§5.3). Condition the DEM (§5.7). Implement the four gates of §5.9.	4–6 days plus calibration data	Gates 1–4 pass. Access distances do not change by more than one cell when the CRS changes.
P3 Make it configurable	Nothing is hard-coded; a run is described by a config file.	Every item in §4.1 becomes a config field with a unit in its name. The <code>RunConfig</code> dataclass and the YAML loader. <code>RunContext</code> replacing the 14- and 9-argument signatures (S3). The explicit error types of <code>errors.py</code> .	3–4 days	A run is fully described by one YAML file and a manifest. No literal path, buffer or threshold remains in the source.
P4 Make it maintainable	The package is a library with tests.	The <code>virtualroads/</code> package from §1.3. The eight tests of §11.3. Delete the orphan output step and add it properly (§10.1). <code>GeoPackage</code> output plus a manifest and a CSV (§10.4). The vectorised <code>array2shp</code> and <code>MEM</code> datasets (§11.1).	5–7 days	<code>pytest</code> green. The regression test pins the 186-vertex path. The national run is one command.
P5 Make it a national product	The model is calibrated for South Africa and the results are publishable.	The relief-zone split and the per-zone parameter sets (§6.2, §6.3). Land-cover traction (§5.4). Road-class selection (§6.5). Seasonality (§6.6). Bridge and protection layers (§5.6, §6.7). Calibrate against surveyed access routes (§5.9). Publish the cost surface, add a datasheet and a licence (§11.6).	— Open-ended, and it is the work that turns a script into a product.	Each relief zone has calibrated parameters with a standard error, a confidence flag, and a published datasheet.

B.1 What not to do

- **Do not keep the “slope only” cost model as the production model.** It minimises total climb, which is not a meaningful objective. Keep it as `cost_model: slope_only only` so that the regression test and previously published results remain reproducible (§5.9 gate 1).
- **Do not run this nationally before P2 is complete.** The zero-cost tile-seam bug (B04) and the anisotropic routing (§5.1) will both produce plausible-looking but wrong results, and neither leaves a visible symptom.
- **Do not keep 880-file output directories.** At national scale this is millions of files, and it will break the file system before it breaks the model.
- **Do not add more parameters before P3.** There are already 20 hard-coded values (§4.1); adding a 21st in place makes the model harder to run, not more flexible.
- **Do not replace scikit-image.** `MCP_Geometric` is the right tool for a grid minimum-cost path. What is needed is a metric spacing, an explicit barrier representation, and — eventually — a state-augmented search for the turn penalty (§5.5). A full reimplementation would be a large, risky change for no analytical gain.